Chapter 10 ▪ Graphical User Interface Programming

Java was born at a time when most computer users interacted with desktop applications via a graphical user interface (GUI). Nowadays, browser-based and mobile applications are far more common, but there are still times when it is useful to provide a desktop application. Moreover, many teachers and students enjoy learning Java through GUI applications. In this and the following chapter, I discuss the basics of user interface programming with the Swing toolkit. **Chapter 12** covers more advanced techniques that can also be useful to generate images on a server. If you are not interested in writing GUI and graphics programs, you can safely skip these chapters.

10.1. A History of Java User Interface Toolkits

When Java 1.0 was introduced, it contained a class library, called the Abstract Window Toolkit (AWT), for basic GUI programming. The basic AWT library deals with user interface elements by delegating their creation and behavior to the native GUI toolkit on each target platform (Windows, Linux, Macintosh, and so on). For example, if you used the original AWT to put a text box on a Java window, an underlying "peer" text box actually handled the text input. The resulting program could then, in theory, run on any of these platforms, with the "look-and-feel" of the target platform.

The peer-based approach worked well for simple applications, but it soon became apparent that it was fiendishly difficult to write a high-quality portable graphics library depending on native user interface elements. User interface elements such as menus, scrollbars, and text fields can have subtle differences in behavior on different platforms. It was hard, therefore, to give users a consistent and predictable experience with this approach. Moreover, some graphical environments (such as X11/Motif) do not have as rich a collection of user interface components as does Windows or the Macintosh. This further limits a portable library based on a "lowest common denominator" approach. As a result, GUI applications built with the AWT simply did not look as nice as native Windows or Macintosh applications, nor did they have the kind of functionality that users of those platforms had come to expect. More depressingly, there were *different* bugs in the AWT user interface library on the different platforms. Developers complained that they had to test their applications on each platform—a practice derisively called "write once, debug everywhere."

In 1996, Netscape created a GUI library they called the IFC (Internet Foundation Classes) that used an entirely different approach. User interface elements, such as buttons, menus, and so on, were *painted* onto blank windows. The only functionality required from the underlying windowing system was a way to put up a window and to paint on it. Thus, Netscape's IFC widgets looked and behaved the same no matter which platform the program ran on. Sun Microsystems worked with Netscape to perfect this approach, creating a user interface library with the code name "Swing." Swing was available as an extension to Java 1.1 and became a part of the standard library in Java 1.2.

Swing is now the official name for the non-peer-based GUI toolkit.



Note: Swing is not a complete replacement for the AWT—it is built on top of the AWT architecture. Swing simply gives you more capable user interface components. Whenever you write a Swing program, you use the foundations of the AWT—in particular, event handling. From now on, I will say "Swing" when I mean

the “painted” user interface classes, and “AWT” when I refer to the underlying mechanisms of the windowing toolkit, such as event handling.

Swing has to work hard painting every pixel of the user interface. When Swing was first released, users complained that it was slow. (You can still get a feel for the problem if you run Swing applications on hardware such as a Raspberry Pi.) After a while, desktop computers got faster, and users complained that Swing was ugly—indeed, it had fallen behind the native widgets that had been spruced up with animations and fancy effects. More ominously, Adobe Flash was increasingly used to create user interfaces with even flashier effects that didn't use the native controls at all.

In 2007, Sun Microsystems introduced an entirely different user interface toolkit, called JavaFX, as a competitor to Flash. It ran on the Java VM but had its own programming language, called JavaFX Script. The language was optimized for programming animations and fancy effects. Programmers complained about the need to learn a new language, and they stayed away in droves. In 2011, Oracle released a new version, JavaFX 2.0, that had a Java API and no longer needed a separate programming language. Starting with Java 7 update 6, JavaFX was bundled with the JDK and JRE. However, this bundling was discontinued starting with Java 11.

Since this is a book about the core Java language and APIs, I will focus on Swing for user interface programming.

10.2. Displaying Frames

A top-level window (that is, a window that is not contained inside another window) is called a *frame* in Java. The AWT library has a class, called `Frame`, for this top level. The Swing version of this class is called `JFrame` and extends the `Frame` class. The `JFrame` is one of the few Swing components that is not painted on a canvas. Thus, the decorations (buttons, title bar, icons, and so on) are drawn by the user’s windowing system, not by Swing.



Caution: Most Swing component classes start with a “J”: `JButton`, `JFrame`, and so on. There are classes such as `Button` and `Frame`, but they are AWT components. If you accidentally omit a “J”, your program may still compile and run, but the mixture of Swing and AWT components can lead to visual and behavioral inconsistencies.

10.2.1. Creating a Frame

In this section, we will go over the most common methods for working with a Swing `JFrame`. **Listing 10.1** lists a simple program that displays an empty frame on the screen, as illustrated in **Figure 10.1**.



Figure 10.1: The simplest visible frame

Listing 10.1 v2ch10/simpleFrame/SimpleFrameDemo.java

```
1  package v2ch10.simpleFrame;
2
3  import module java.desktop;
4
```

```
class SimpleFrameDemo {
    void main() {
        EventQueue.invokeLater(() -> {
            var frame = new SimpleFrame();
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setVisible(true);
        });
    }
}

class SimpleFrame extends JFrame {
    private static final int DEFAULT_WIDTH = 300;
    private static final int DEFAULT_HEIGHT = 200;

    public SimpleFrame() {
        setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);
    }
}
```

Let's work through this program, line by line.

The Swing classes are placed in the `javax.swing` package. The package name `java`*`x`* indicates a Java extension package, not a core package. For historical reasons, Swing is considered an extension. However, it is present in every Java implementation since version 1.2.

By default, a frame has a rather useless size of 0×0 pixels. We define a subclass `SimpleFrame` whose constructor sets the size to 300×200 pixels. This is the only difference between a `SimpleFrame` and a `JFrame`.

In the `main` method of the `SimpleFrameDemo` class, we construct a `SimpleFrame` object and make it visible.

There are two technical issues that we need to address in every Swing program.

First, all Swing components must be configured from the *event dispatch thread*, the thread of control that passes events such as mouse clicks and keystrokes to the user interface components. The following code fragment is used to execute statements in the event dispatch thread:

```
EventQueue.invokeLater(() -> {
    statements
});
```



Note: You will see many Swing programs that do not initialize the user interface in the event dispatch thread. It used to be perfectly acceptable to carry out the initialization in the main thread. Sadly, as Swing components got more complex, the developers of the JDK were no longer able to guarantee the safety of that approach. The probability of an error is extremely low, but you would not want to be one of the unlucky few who encounter an intermittent problem. It is better to do the right thing, even if the code looks rather mysterious.

Next, we define what should happen when the user closes the application's frame. For this particular program, we want the program to exit. To select this behavior, we use the statement

```
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

In other programs with multiple frames, you would not want the program to exit just because the user closed one of the frames. By default, a frame is hidden when the user closes it, but the program does not terminate. (It might have been nice if the program terminated once the *last* frame becomes invisible, but that is not how Swing works.)

Simply constructing a frame does not automatically display it. Frames start their life invisible. That gives the programmer the chance to add components into the frame before showing it for the first time. To show the frame, the `main` method calls the `setVisible` method of the frame.

After scheduling the initialization statements, the `main` method exits. Note that exiting `main` does not terminate the program—just the main thread. The event dispatch thread keeps the program alive until it is terminated, either by closing the frame or by calling the `System.exit` method.

The running program is shown in **Figure 10.1**—it is a truly boring top-level window. As you can see in the figure, the title bar and the surrounding decorations, such as resize corners, are drawn by the operating system and not the Swing library. The Swing library draws everything inside the frame. In this program, it just fills the frame with a default background color.

10.2.2. Frame Properties

The `JFrame` class itself has only a few methods for changing how frames look. Of course, through the magic of inheritance, most of the methods for working with the size and position of a frame come from the various superclasses of `JFrame`. Here are some of the most important methods:

- The `setLocation` and `setBounds` methods for setting the position of the frame
- The `setIconImage` method, which tells the windowing system which icon to display in the title bar, task switcher window, and so on
- The `setTitle` method for changing the text in the title bar
- The `setResizable` method, which takes a `boolean` to determine if a frame will be resizeable by the user

Figure 10.2 illustrates the inheritance hierarchy for the `JFrame` class.

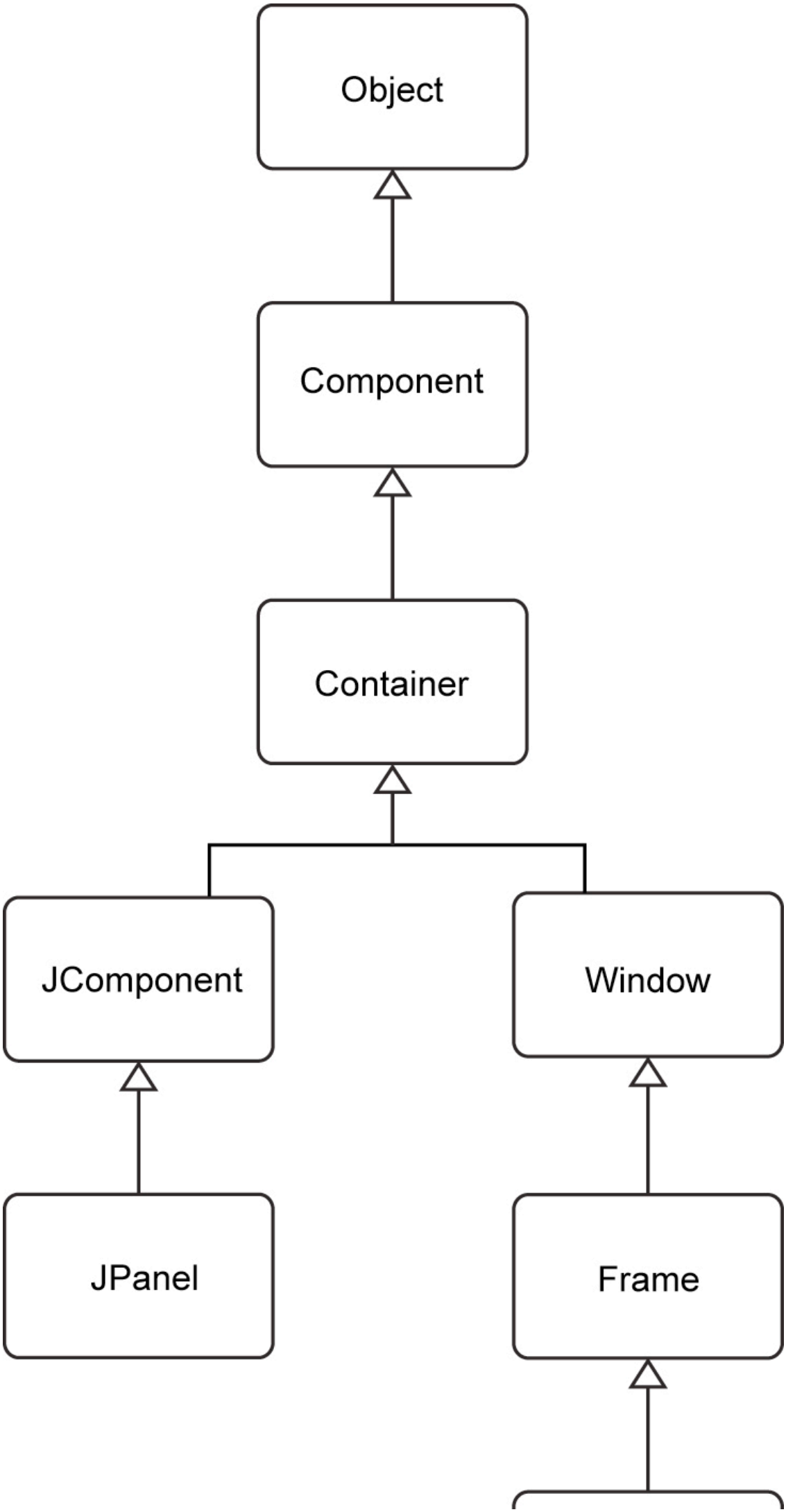

Object
Component
Container
JComponent
Window
JPanel
Frame



Figure 10.2: Inheritance hierarchy for the frame and component classes in AWT and Swing

As the API notes indicate, the `Component` class (which is the ancestor of all GUI objects) and the `Window` class (which is the superclass of the `Frame` class) are where you need to look for the methods to resize and reshape frames. For example, the `setLocation` method in the `Component` class is one way to reposition a component. If you make the call

```
setLocation(x, y)
```

the top left corner is located `x` pixels across and `y` pixels down, where (0, 0) is the top left corner of the screen. Similarly, the `setBounds` method in `Component` lets you resize and relocate a component (in particular, a `JFrame`) in one step, as

```
setBounds(x, y, width, height)
```

Many methods of component classes come in getter/setter pairs, such as the following methods of the `Frame` class:

```
public String getTitle()
public void setTitle(String title)
```

Such a getter/setter pair is called a *property*. A property has a name and a type. The name is obtained by changing the first letter after the `get` or `set` to lowercase. For example, the `Frame` class has a property with name `title` and type `String`.

Conceptually, `title` is a property of the frame. When we set the property, we expect the title to change on the user's screen. When we get the property, we expect to get back the value that we have set.

There is one exception to the `get`/`set` convention: For properties of type `boolean`, the getter starts with `is`. For example, the following two methods define the `resizable` property:

```
public boolean isResizable()
public void setResizable(boolean resizable)
```

To determine an appropriate size for a frame, first find out the screen size. Call the static `getDefaultToolkit` method of the `Toolkit` class to get the `Toolkit` object. (The `Toolkit` class is a dumping ground for a variety of methods interfacing with the native windowing system.) Then call the `getScreenSize` method, which returns the screen size as a `Dimension` object. A `Dimension` object simultaneously stores a width and a height, in public (!) instance fields `width` and `height`. Then you can use a suitable percentage of the screen size to size the frame. Here is the code:

```
Toolkit kit = Toolkit.getDefaultToolkit();
Dimension screenSize = kit.getScreenSize();
int screenWidth = screenSize.width;
int screenHeight = screenSize.height;
setSize(screenWidth / 2, screenHeight / 2);
```

You can also supply a frame icon:

```
Image img = new ImageIcon("icon.gif").getImage();
setIconImage(img);
```

`java.awt.Component` 1.0

- `boolean isVisible()`
- `void setVisible(boolean b)`
  get or set the `visible` property. Components are initially visible, with the exception of top-level components such as `JFrame`.
- `void setSize(int width, int height)` 1.1
  resizes the component to the specified width and height.
- `void setLocation(int x, int y)` 1.1
  moves the component to a new location. The `x` and `y` coordinates use the coordinates of the container if the component is not a top-level component, or the coordinates of the screen if the component is top level (for example, a `JFrame`).
- `void setBounds(int x, int y, int width, int height)` 1.1
  moves and resizes this component.
- `Dimension getSize()` 1.1
- `void setSize(Dimension d)` 1.1
  get or set the `size` property of this component.

`java.awt.Frame` 1.0

- `boolean isResizable()`
- `void setResizable(boolean b)`
  get or set the `resizable` property. When the property is set, the user can resize the frame.
- `String getTitle()`
- `void setTitle(String s)`
  get or set the `title` property that determines the text in the title bar for the frame.
- `Image getIconImage()`
- `void setIconImage(Image image)`
  get or set the `iconImage` property that determines the icon for the frame. The windowing system may display the icon as part of the frame decoration or in other locations.

`java.awt.Toolkit` 1.0

- `static Toolkit getDefaultToolkit()`
  returns the default toolkit for this platform.
- `Dimension getScreenSize()`
  gets the size of the user's screen.

`javax.swing.ImageIcon` 1.2

- `ImageIcon(String filename)`
  constructs an icon whose image is stored in a file.
- `Image getImage()`
  gets the image of this icon.

10.3. Displaying Information in a Component

In this section, I will show you how to display information inside a frame (**Figure 10.3**).



Figure 10.3: A frame that displays information

You could draw the message string directly onto a frame, but that is not considered good programming practice. In Java, frames are really designed to be containers for components, such as a menu bar and other user interface elements. You normally draw on another component which you add to the frame.

The structure of a `JFrame` is surprisingly complex. Look at **Figure 10.4** which shows the makeup of a `JFrame`. As you can see, four panes are layered in a `JFrame`. The root pane, layered pane, and glass pane are of no interest to us; they are required to organize the menu bar and content pane and to implement the look-and-feel. The part that most concerns Swing programmers is the *content pane*. Any components that you add to a frame are automatically placed into the content pane:

```
Component c = . . .;
frame.add(c); // added to the content pane
```

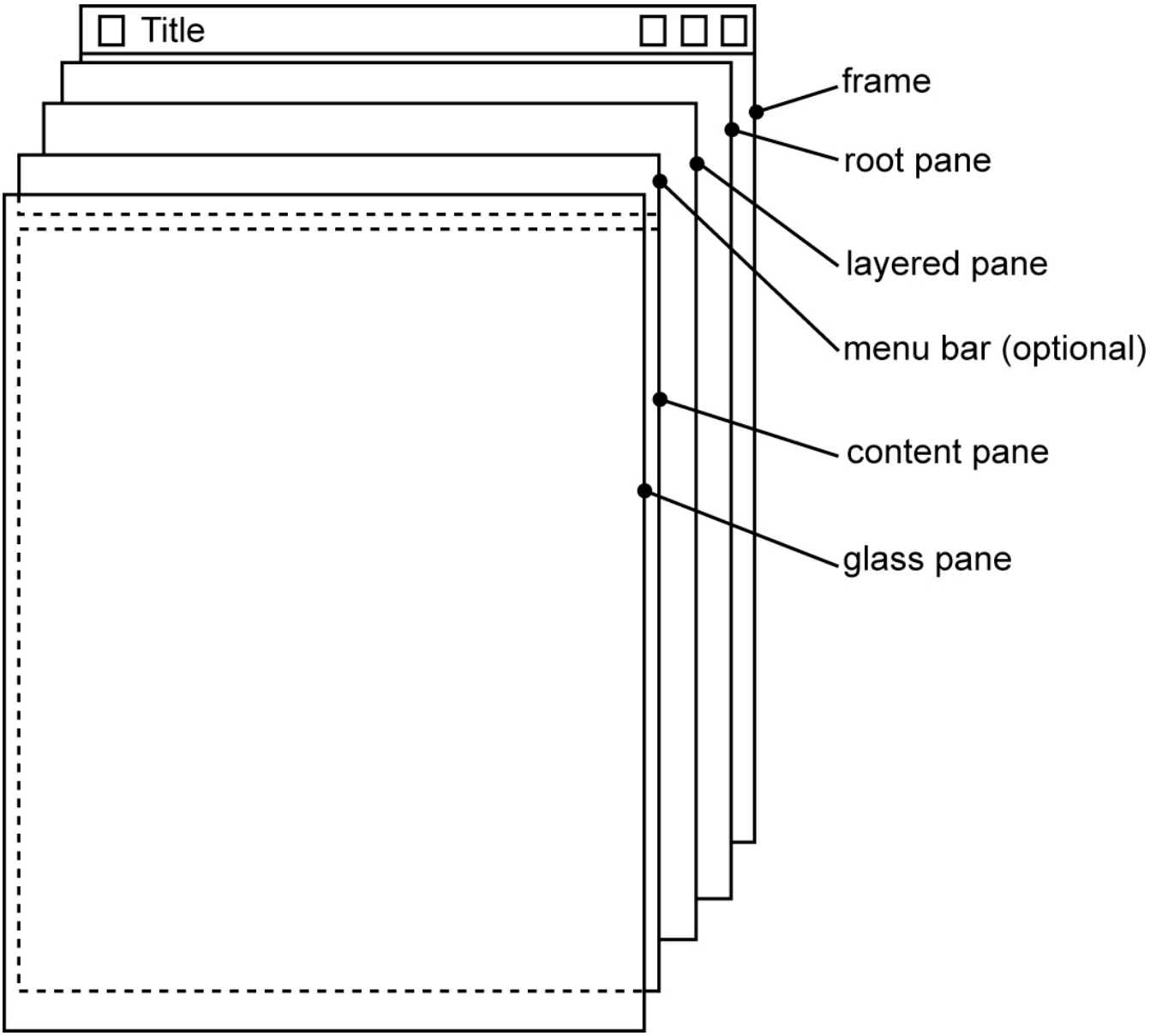


Figure 10.4: Internal structure of a JFrame

In our case, we want to add a single component to the frame onto which we will draw our message. To draw on a component, you define a class that extends `JComponent` and override the `paintComponent` method in that class.

The `paintComponent` method has one parameter of type `Graphics`. A `Graphics` object remembers a collection of settings for drawing images and text, such as the font you set or the current color. All drawing in Java must go through a `Graphics` object. It has methods that draw patterns, images, and text.

Here's how to make a component onto which you can draw:

```
class MyComponent extends JComponent {
    public void paintComponent(Graphics g) {
        code for drawing
    }
}
```

Each time a window needs to be redrawn, no matter what the reason, the event handler notifies the component. This causes the `paintComponent` methods of all components to be executed.

Never call the `paintComponent` method yourself. It is called automatically whenever a part of your application needs to be redrawn, and you should not interfere with this automatic process.

What sorts of actions trigger this automatic response? For example, painting occurs when the user increases the size of the window, or minimizes and then restores the window. If the user popped up another window that covered an existing window and then made the overlaid window disappear, the window that was covered is now corrupted and will need to be repainted. (The graphics system does not save the pixels underneath.) And, of course, when the window is displayed for the first time, it needs to process the code that specifies how and where it should draw the initial elements.



Tip: If you need to force repainting of the screen, call the `repaint` method instead of `paintComponent`. The `repaint` method will cause `paintComponent` to be called for all components, with a properly configured `Graphics` object.

As you saw in the code fragment above, the `paintComponent` method has a single parameter of type `Graphics`. Measurement on a `Graphics` object for screen display is done in pixels. The (0, 0) coordinate denotes the top left corner of the component on whose surface you are drawing.

The `Graphics` class has various drawing methods, and displaying text is considered a special kind of drawing. Our `paintComponent` method looks like this:

```
public class NotHelloWorldComponent extends JComponent {
    public static final int MESSAGE_X = 75;
    public static final int MESSAGE_Y = 100;

    public void paintComponent(Graphics g) {
        g.drawString("Not a Hello, World program", MESSAGE_X, MESSAGE_Y);
    }
    . . .
}
```

Finally, a component should tell its users how big it would like to be. Override the `getPreferredSize` method and return an object of the `Dimension` class with the preferred width and height:

```
public class NotHelloWorldComponent extends JComponent {
    private static final int DEFAULT_WIDTH = 300;
    private static final int DEFAULT_HEIGHT = 200;
    . . .
    public Dimension getPreferredSize() {
        return new Dimension(DEFAULT_WIDTH, DEFAULT_HEIGHT);
    }
}
```

When you fill a frame with one or more components, and you simply want to use their preferred size, call the `pack` method instead of the `setSize` method:

```
class NotHelloWorldFrame extends JFrame {
    public NotHelloWorldFrame() {
        add(new NotHelloWorldComponent());
```

```
        pack();
    }
}
```

Listing 10.2 shows the complete code.

Listing 10.2 v2ch10/notHelloWorld/NotHelloWorld.java

```
1  package v2ch10.notHelloWorld;
2
3  import module java.desktop;
4
5  class NotHelloWorld {
6      void main() {
7          EventQueue.invokeLater(() -> {
8              var frame = new NotHelloWorldFrame();
9              frame.setTitle("NotHelloWorld");
10             frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
11             frame.setVisible(true);
12         });
13     }
14 }
15
16 /**
17  * A frame that contains a message panel.
18  */
19 class NotHelloWorldFrame extends JFrame {
20     public NotHelloWorldFrame() {
21         add(new NotHelloWorldComponent());
22         pack();
23     }
24 }
25
26 /**
27  * A component that displays a message.
28  */
29 class NotHelloWorldComponent extends JComponent {
30     public static final int MESSAGE_X = 75;
31     public static final int MESSAGE_Y = 100;
32
33     private static final int DEFAULT_WIDTH = 300;
34     private static final int DEFAULT_HEIGHT = 200;
35
36     public void paintComponent(Graphics g) {
37         g.drawString("Not a Hello, World program", MESSAGE_X, MESSAGE_Y);
38     }
39
40     public Dimension getPreferredSize() {
41         return new Dimension(DEFAULT_WIDTH, DEFAULT_HEIGHT);
42     }
43 }
```

`javax.swing.JFrame` 1.2

- `Component add(Component c)`
  adds and returns the given component to the content pane of this frame.

`java.awt.Component` 1.0

- `void repaint()`
  causes a repaint of the component "as soon as possible."
- `Dimension getPreferredSize()`
  is the method to override to return the preferred size of this component.

`javax.swing.JComponent` 1.2

- `void paintComponent(Graphics g)`
  is the method to override to describe how your component needs to be painted.

`java.awt.Window` 1.0

- `void pack()`
  resizes this window, taking into account the preferred sizes of its components.

10.3.1. Working with 2D Shapes

Starting with Java 1.0, the `Graphics` class has methods to draw lines, rectangles, ellipses, and so on. But those drawing operations are very limited. We will instead use the shape classes from the *Java 2D* library.

To use this library, you need to obtain an object of the `Graphics2D` class. This class is a subclass of the `Graphics` class. Ever since Java 1.2, methods such as `paintComponent` automatically receive an object of the `Graphics2D` class. Simply use a cast, as follows:

```
public void paintComponent(Graphics g) {
    Graphics2D g2 = (Graphics2D) g;
    . . .
}
```

The Java 2D library organizes geometric shapes in an object-oriented fashion. In particular, there are classes to represent lines, rectangles, and ellipses:

```
Line2D
Rectangle2D
Ellipse2D
```

These classes all implement the `Shape` interface. The Java 2D library supports more complex shapes—arcs, quadratic and cubic curves, and general paths—that we do not discuss in this chapter.

To draw a shape, you first create an object of a class that implements the `Shape` interface and then call the `draw` method of the `Graphics2D` class. For example:

```
Rectangle2D rect = . . .;
g2.draw(rect);
```

The Java 2D library uses floating-point coordinates, not integers, for pixels. Internal calculations are carried out with single-precision `float` quantities. Single precision is sufficient—after all, the ultimate purpose of the geometric computations is to set pixels on the screen or printer. As long as any roundoff errors stay within one pixel, the visual outcome is not affected.

However, manipulating `float` values is sometimes inconvenient for the programmer because Java is adamant about requiring casts when converting `double` values into `float` values. For example, consider the following statement:

```
float f = 1.2; // ERROR--possible loss of precision
```

This statement does not compile because the constant `1.2` has type `double`, and the compiler is nervous about loss of precision. The remedy is to add an `F` suffix to the floating-point constant:

```
float f = 1.2F; // OK
```

Now consider this statement:

```
float f = r.getWidth(); // ERROR
```

This statement does not compile either, for the same reason. The `getWidth` method returns a `double`. This time, the remedy is to provide a cast:

```
float f = (float) r.getWidth(); // OK
```

These suffixes and casts are a bit of a pain, so the designers of the 2D library decided to supply *two versions* of each shape class: one with `float` coordinates for frugal programmers, and one with `double` coordinates for the lazy ones. (In this book, we fall into the second camp and use `double` coordinates whenever we can.)

The library designers chose a curious mechanism for packaging these choices. Consider the `Rectangle2D` class. This is an abstract class with two concrete subclasses, which are also static nested classes:

```
Rectangle2D.Float
Rectangle2D.Double
```

Figure 10.5 shows the inheritance diagram.

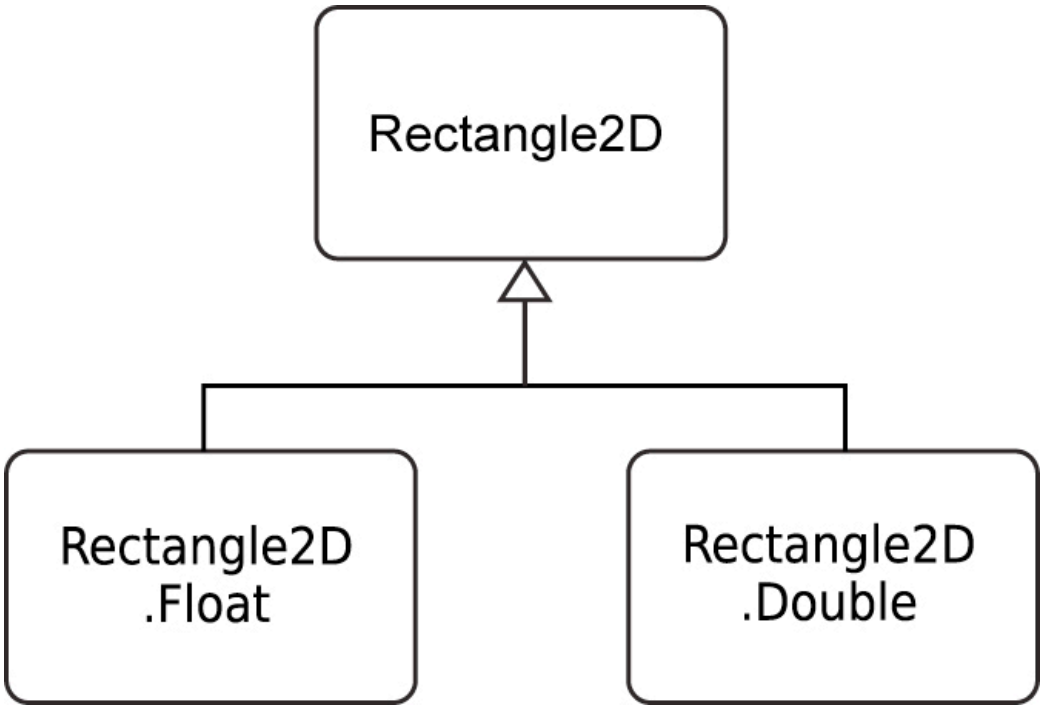


Figure 10.5: 2D rectangle classes

It is best to ignore the fact that the two concrete classes are static inner classes—that is just a gimmick to avoid names such as `FloatRectangle2D` and `DoubleRectangle2D`.

When you construct a `Rectangle2D.Float` object, you supply the coordinates as `float` numbers. For a `Rectangle2D.Double` object, you supply them as `double` numbers.

```
var floatRect = new Rectangle2D.Float(10.0F, 25.0F, 22.5F, 20.0F);
var doubleRect = new Rectangle2D.Double(10.0, 25.0, 22.5, 20.0);
```

The construction arguments denote the top left corner, width, and height of the rectangle.

The `Rectangle2D` methods have `double` parameter and return types. For example, the `getWidth` method returns a `double` value, even if the width is stored as a `float` in a `Rectangle2D.Float` object.



Tip: Simply use the `Double` shape classes to avoid dealing with `float` values altogether. However, if you are constructing thousands of shape objects, consider using the `Float` classes to conserve memory.

What we just discussed for the `Rectangle2D` classes holds for the other shape classes as well. Furthermore, there is a `Point2D` class with subclasses `Point2D.Float` and `Point2D.Double`. Here is how to make a point object:

```
var p = new Point2D.Double(10, 20);
```

The classes `Rectangle2D` and `Ellipse2D` both inherit from the common superclass `RectangularShape`. Admittedly, ellipses are not rectangular, but they have a *bounding rectangle* (see **Figure 10.6**).

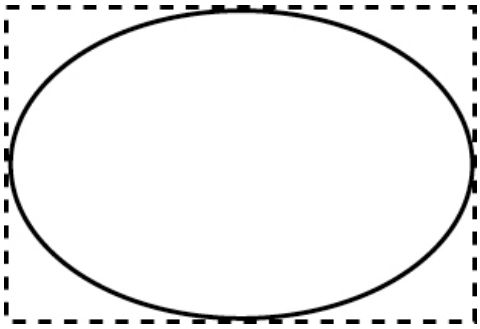


Figure 10.6: The bounding rectangle of an ellipse

The `RectangularShape` class defines over 20 methods that are common to these shapes, among them such useful methods as `getWidth`, `getHeight`, `getCenterX`, and `getCenterY` (but, sadly, at the time of this writing, not a `getCenter` method that would return the center as a `Point2D` object).

Finally, a couple of legacy classes from Java 1.0 have been fitted into the shape class hierarchy. The `Rectangle` and `Point` classes, which store a rectangle and a point with integer coordinates, extend the `Rectangle2D` and `Point2D` classes.

Figure 10.7 shows the relationships between the shape classes. However, the `Double` and `Float` subclasses are omitted. Legacy classes are marked with a gray fill.

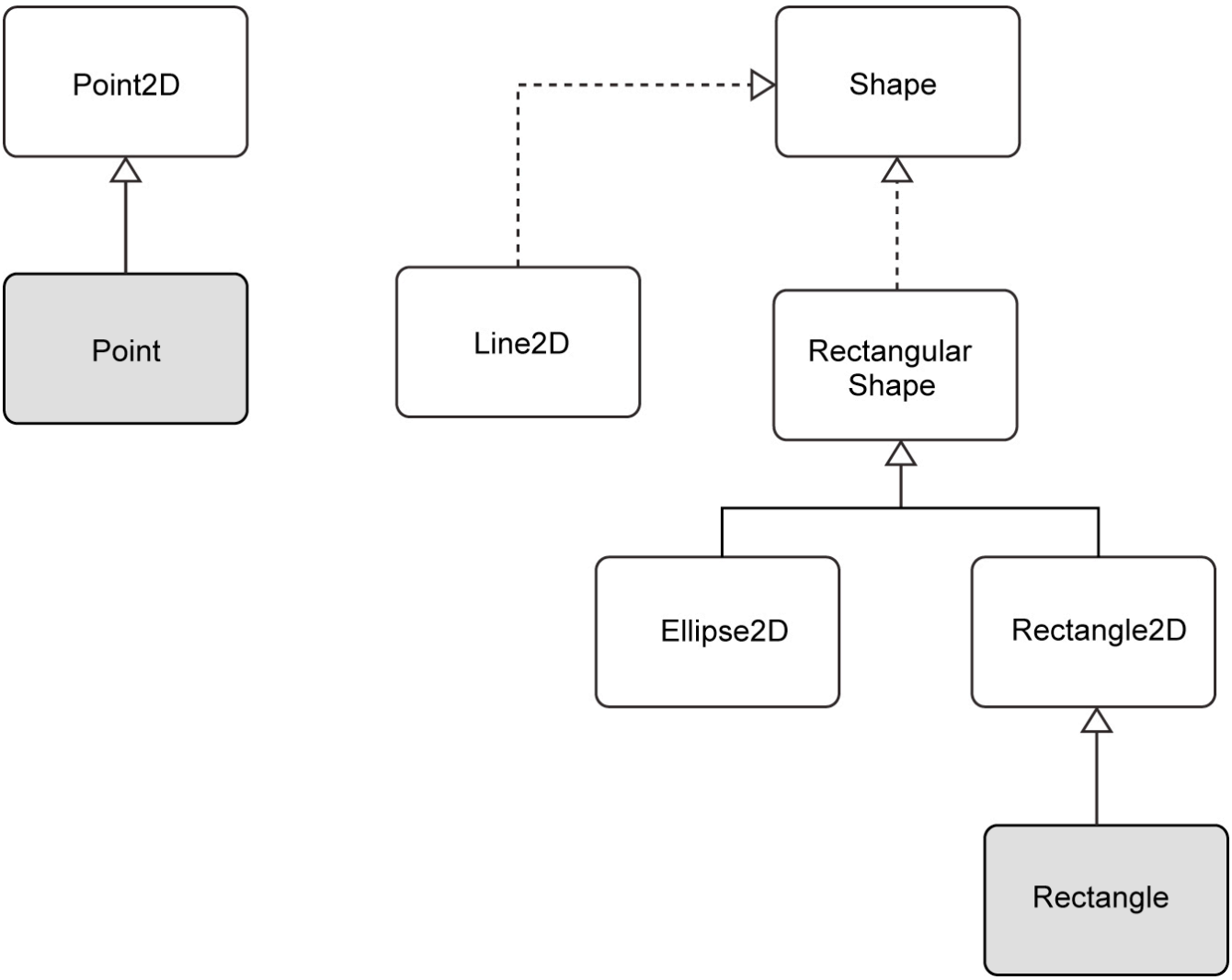


Figure 10.7: Relationships between the shape classes

`Rectangle2D` and `Ellipse2D` objects are simple to construct. You need to specify

- The *x* and *y* coordinates of the top left corner

- The width and height

For ellipses, these refer to the bounding rectangle. For example,

```
var e = new Ellipse2D.Double(150, 200, 100, 50);
```

constructs an ellipse that is bounded by a rectangle with the top left corner at (150, 200), width of 100, and height of 50.

When constructing an ellipse, you usually know the center, width, and height, but not the corner points of the bounding rectangle (which don't even lie on the ellipse). The `setFrameFromCenter` method uses the center point, but it still requires one of the four corner points. Thus, you will usually end up constructing an ellipse as follows:

```
var ellipse =
    new Ellipse2D.Double(centerX - width / 2, centerY - height / 2, width, height);
```

To construct a line, you supply the start and end points, either as `Point2D` objects or as pairs of numbers:

```
var line = new Line2D.Double(start, end);
```

or

```
var line = new Line2D.Double(startX, startY, endX, endY);
```

The program in **Listing 10.3** draws a rectangle, the ellipse that is enclosed in the rectangle, a diagonal of the rectangle, and a circle that has the same center as the rectangle. **Figure 10.8** shows the result.

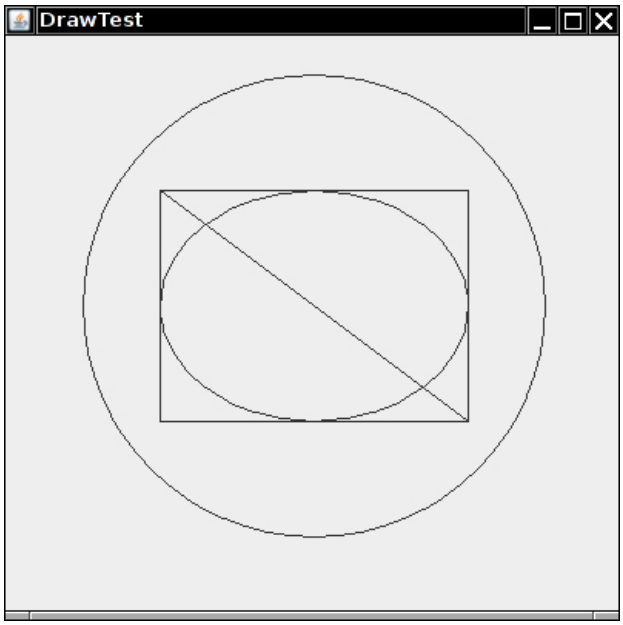


Figure 10.8: Drawing geometric shapes

Listing 10.3 v2ch10/draw/DrawDemo.java

```
package v2ch10.draw;

import module java.desktop;

class DrawDemo {
    void main() {
        EventQueue.invokeLater(() -> {
            var frame = new DrawFrame();
            frame.setTitle("DrawTest");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setVisible(true);
        });
    }
}

/**
 * A frame that contains a panel with drawings.
 */
class DrawFrame extends JFrame {
    public DrawFrame() {
        add(new DrawComponent());
        pack();
    }
}

/**
 * A component that displays rectangles and ellipses.
 */
class DrawComponent extends JComponent {
    private static final int DEFAULT_WIDTH = 400;
    private static final int DEFAULT_HEIGHT = 400;

    public void paintComponent(Graphics g) {
        var g2 = (Graphics2D) g;

        // draw a rectangle

        double leftX = 100;
        double topY = 100;
        double width = 200;
        double height = 150;

        var rect = new Rectangle2D.Double(leftX, topY, width, height);
        g2.draw(rect);

        // draw the enclosed ellipse

        var ellipse = new Ellipse2D.Double();
        ellipse.setFrame(rect);
        g2.draw(ellipse);

        // draw a diagonal line

        g2.draw(new Line2D.Double(leftX, topY, leftX + width, topY + height));

        // draw a circle with the same center
```

```
57
58          double centerX = rect.getCenterX();
59          double centerY = rect.getCenterY();
60          double radius = 150;
61
62          var circle = new Ellipse2D.Double();
63          circle.setFrameFromCenter(centerX, centerY, centerX + radius, centerY + radius);
64          g2.draw(circle);
65      }
66
67      public Dimension getPreferredSize() {
68          return new Dimension(DEFAULT_WIDTH, DEFAULT_HEIGHT);
69      }
70  }
```

`java.awt.geom.RectangularShape` 1.2

- `double getCenterX()`
- `double getCenterY()`
- `double getMinX()`
- `double getMinY()`
- `double getMaxX()`
- `double getMaxY()`

  return the center, minimum, or maximum *x* or *y* value of the enclosing rectangle.
- `double getWidth()`
- `double getHeight()`

  return the width or height of the enclosing rectangle.
- `double getX()`
- `double getY()`

  return the *x* or *y* coordinate of the top left corner of the enclosing rectangle.

`java.awt.geom.Rectangle2D.Double` 1.2

- `Rectangle2D.Double(double x, double y, double w, double h)`

  constructs a rectangle with the given top left corner, width, and height.

`java.awt.geom.Ellipse2D.Double` 1.2

- `Ellipse2D.Double(double x, double y, double w, double h)`

  constructs an ellipse whose bounding rectangle has the given top left corner, width, and height.

`java.awt.geom.Point2D.Double` 1.2

- `Point2D.Double(double x, double y)`

  constructs a point with the given coordinates.

`java.awt.geom.Line2D.Double` 1.2

- `Line2D.Double(Point2D start, Point2D end)`
- `Line2D.Double(double startX, double startY, double endX, double endY)`

construct a line with the given start and end points.

10.3.2. Using Color

The `setPaint` method of the `Graphics2D` class lets you select a color that is used for all subsequent drawing operations on the graphics context. For example:

```
g2.setPaint(Color.RED);
g2.drawString("Warning!", 100, 100);
```

You can fill the interiors of closed shapes (such as rectangles or ellipses) with a color. Simply call `fill` instead of `draw`:

```
Rectangle2D rect = . . .;
g2.setPaint(Color.RED);
g2.fill(rect); // fills rect with red
```

To draw in multiple colors, select a color, draw or fill, then select another color, and draw or fill again.



Note: The `fill` method paints one fewer pixel to the right and the bottom. For example, if you draw a `new Rectangle2D.Double(0, 0, 10, 20)`, then the drawing includes the pixels with $x = 10$ and $y = 20$. If you fill the same rectangle, those pixels are not painted.

Define colors with the `Color` class. The `java.awt.Color` class offers predefined constants for the following 13 standard colors:

```
BLACK, BLUE, CYAN, DARK_GRAY, GRAY, GREEN, LIGHT_GRAY,
MAGENTA, ORANGE, PINK, RED, WHITE, YELLOW
```

You can specify a custom color by creating a `Color` object by its red, green, and blue components, each a value between `0` and `255`:

```
g2.setPaint(new Color(0, 128, 128)); // a dull blue-green
g2.drawString("Welcome!", 75, 125);
```



Note: In addition to solid colors, you can call `setPaint` with instances of classes that implement the `Paint` interface. This enables drawing with gradients and textures.

To set the background color of a component, use the `setBackground` method of the `Component` class, an ancestor of `JComponent`.

```
var component = new MyComponent();
component.setBackground(Color.PINK);
```

There is also a `setForeground` method. It specifies the default color that is used for drawing on the component.

`java.awt.Color` 1.0

- `Color(int r, int g, int b)`
  creates a color object with the given red, green, and blue components between `0` and `255`.

`java.awt.Graphics2D` 1.2

- `Paint getPaint()`
- `void setPaint(Paint p)`
  get or set the paint property of this graphics context. The `Color` class implements the `Paint` interface. Therefore, you can use this method to set the paint attribute to a solid color.
- `void fill(Shape s)`
  fills the shape with the current paint.

`java.awt.Component` 1.0

- `Color getForeground()`
- `Color getBackground()`
- `void setForeground(Color c)`
- `void setBackground(Color c)`
  get or set the foreground or background color.

10.3.3. Using Fonts

The "Not a Hello World" program at the beginning of this chapter displayed a string in the default font. Sometimes, you will want to show your text in a different font. You can specify a font by its *font face name*. A font face name is composed of a *font family name*, such as "Helvetica," and an optional suffix such as "Bold." For example, the font faces "Helvetica" and "Helvetica Bold" are both considered to be part of the family named "Helvetica."

To find out which fonts are available on a particular computer, call the `getAvailableFontFamilyNames` method of the `GraphicsEnvironment` class. The method returns an array of strings containing the names of all available fonts. To obtain an instance of the `GraphicsEnvironment` class that describes the graphics environment of the user's system, use the static `getLocalGraphicsEnvironment` method. The following program prints the names of all fonts on your system:

```
import java.awt.*;

void main() {
    String[] fontNames = GraphicsEnvironment
        .getLocalGraphicsEnvironment()
        .getAvailableFontFamilyNames();
    for (String fontName : fontNames)
        IO.println(fontName);
}
```

The AWT defines five *logical* font names:

```
SansSerif
Serif
Monospaced
Dialog
DialogInput
```

These names are always mapped to some fonts that actually exist on the client machine. For example, on a Windows system, `SansSerif` is mapped to Arial.

In addition, the Oracle JDK always includes three font families named “Lucida Sans,” “Lucida Bright,” and “Lucida Sans Typewriter.”

To draw characters in a font, you must first create an object of the class `Font`. Specify the font face name, the font style, and the point size. Here is an example of how you construct a `Font` object:

```
var sansbold14 = new Font("SansSerif", Font.BOLD, 14);
```

The third argument is the point size. Points are commonly used in typography to indicate the size of a font. There are 72 points per inch.

You can use a logical font name in place of the font face name in the `Font` constructor. Specify the style (plain, **bold**, *italic*, or ***bold italic***) by setting the second `Font` constructor argument to one of the following values:

```
Font.PLAIN
Font.BOLD
Font.ITALIC
Font.BOLD + Font.ITALIC
```

Here’s the code that displays the string “Hello, World!” in the standard sans serif font on your system, using 14-point bold type:

```
var sansbold14 = new Font("SansSerif", Font.BOLD, 14);
g2.setFont(sansbold14);
var message = "Hello, World!";
g2.drawString(message, 75, 100);
```

Next, let’s *center* the string in its component instead of drawing it at an arbitrary position. We need to know the width and height of the string in pixels. These dimensions depend on three factors:

- The font used (in our case, sans serif, bold, 14 point);
- The string (in our case, “Hello, World!”); and
- The device on which the font is drawn (in our case, the user’s screen).

To obtain an object that represents the font characteristics of the screen device, call the `getFontRenderContext` method of the `Graphics2D` class. It returns an object of the `FontRenderContext` class. Simply pass that object to the `getStringBounds` method of the `Font` class:

```
FontRenderContext context = g2.getFontRenderContext();
Rectangle2D bounds = sansbold14.getStringBounds(message, context);
```

The `getStringBounds` method returns a rectangle that encloses the string.

To interpret the dimensions of that rectangle, you should know some basic typesetting terms (see **Figure 10.9**). The *baseline* is the imaginary line where, for example, the bottom of a character like 'e' rests. The *ascent* is the distance from the baseline to the top of an *ascender*, which is the upper part of a letter like 'b' or 'k', or an uppercase character. The *descent* is the distance from the baseline to a *descender*, which is the lower portion of a letter like 'p' or 'g'.

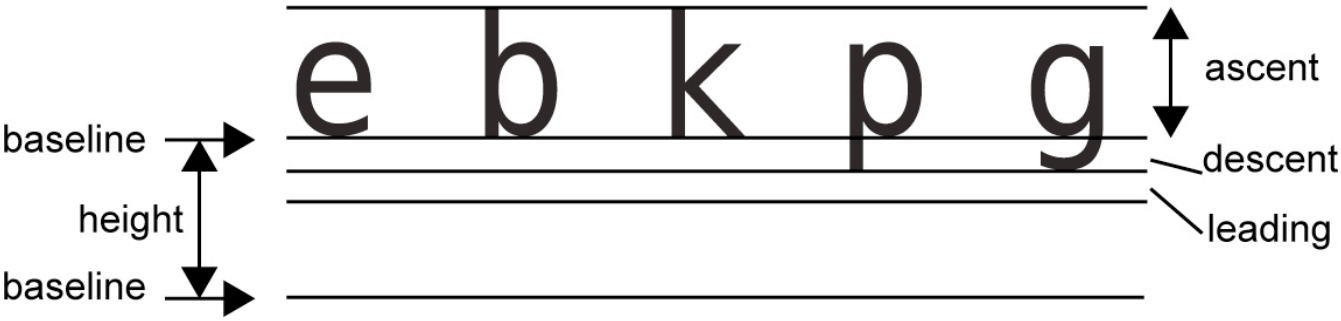


Figure 10.9: Typesetting terms illustrated

Leading is the space between the descent of one line and the ascent of the next line. (The term has its origin from the strips of lead that typesetters used to separate lines.) The *height* of a font is the distance between successive baselines, which is the same as descent + leading + ascent.

The width of the rectangle that the `getStringBounds` method returns is the horizontal extent of the string. The height of the rectangle is the sum of ascent, descent, and leading. The rectangle has its origin at the baseline of the string. The top *y* coordinate of the rectangle is negative. Thus, you can obtain string width, height, and ascent as follows:

```
double stringWidth = bounds.getWidth();
double stringHeight = bounds.getHeight();
double ascent = -bounds.getY();
```

If you need to know the descent or leading, use the `getLineMetrics` method of the `Font` class. That method returns an object of the `LineMetrics` class, which has methods to obtain the descent and leading:

```
LineMetrics metrics = f.getLineMetrics(message, context);
float descent = metrics.getDescent();
float leading = metrics.getLeading();
```



Note: When you need to compute layout dimensions outside the `paintComponent` method, you can't obtain the font render context from the `Graphics2D` object. Instead, call the `getFontMetrics` method of the `JComponent` class and then call `getFontRenderContext`.

```
FontRenderContext context = getFontMetrics(f).getFontRenderContext();
```

To show that the positioning is accurate, the sample program in **Listing 10.4** centers the string in the frame and draws the baseline and the bounding rectangle. **Figure 10.10** shows the screen display.



Figure 10.10: Drawing the baseline and string bounds

Listing 10.4 v2ch10/font/FontDemo.java

```
1  package v2ch10.font;
2
3  import module java.desktop;
4
5  class FontDemo {
6      void main() {
7          EventQueue.invokeLater(() -> {
8              var frame = new FontFrame();
9              frame.setTitle("FontTest");
10             frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
11             frame.setVisible(true);
12         });
13     }
14 }
15
16 /**
17  * A frame with a text message component.
18  */
19 class FontFrame extends JFrame {
20     public FontFrame() {
21         add(new FontComponent());
22         pack();
23     }
24 }
25
26 /**
27  * A component that shows a centered message in a box.
28  */
29 class FontComponent extends JComponent {
30     private static final int DEFAULT_WIDTH = 300;
31     private static final int DEFAULT_HEIGHT = 200;
32
33     public void paintComponent(Graphics g) {
34         var g2 = (Graphics2D) g;
35
36         String message = "Hello, World!";
37
38         var f = new Font("Serif", Font.BOLD, 36);
39         g2.setFont(f);
40
41         // measure the size of the message
42
43         FontRenderContext context = g2.getFontRenderContext();
44         Rectangle2D bounds = f.getStringBounds(message, context);
45
```

```
46          // set (x,y) = top left corner of text
47
48          double x = (getWidth() - bounds.getWidth()) / 2;
49          double y = (getHeight() - bounds.getHeight()) / 2;
50
51          // add ascent to y to reach the baseline
52
53          double ascent = -bounds.getY();
54          double baseY = y + ascent;
55
56          // draw the message
57
58          g2.drawString(message, (int) x, (int) baseY);
59
60          g2.setPaint(Color.LIGHT_GRAY);
61
62          // draw the baseline
63
64          g2.draw(new Line2D.Double(x, baseY, x + bounds.getWidth(), baseY));
65
66          // draw the enclosing rectangle
67
68          var rect = new Rectangle2D.Double(x, y, bounds.getWidth(), bounds.getHeight());
69          g2.draw(rect);
70      }
71
72      public Dimension getPreferredSize() {
73          return new Dimension(DEFAULT_WIDTH, DEFAULT_HEIGHT);
74      }
75  }
```

`java.awt.Font` 1.0

- `Font(String name, int style, int size)`
  creates a new font object. The font name is either a font face name (such as `"Helvetica Bold"`) or a logical font name (such as `"Serif"`, `"SansSerif"`). The style is one of `Font.PLAIN`, `Font.BOLD`, `Font.ITALIC`, or `Font.BOLD + Font.ITALIC`.
- `String getFontName()`
  gets the font face name (such as `"Helvetica Bold"`).
- `String getFamily()`
  gets the font family name (such as `"Helvetica"`).
- `String getName()`
  gets the logical name (such as `"SansSerif"`) if the font was created with a logical font name; otherwise, gets the font face name.
- `Rectangle2D getStringBounds(String s, FontRenderContext context)` 1.2
  returns a rectangle that encloses the string. The origin of the rectangle falls on the baseline. The top *y* coordinate of the rectangle equals the negative of the ascent. The height of the rectangle equals the sum of ascent, descent, and leading. The width equals the string width.
- `LineMetrics getLineMetrics(String s, FontRenderContext context)` 1.2
  returns a line metrics object to determine the extent of the string.

`java.awt.font.LineMetrics` 1.2

- `float getAscent()`
  gets the font ascent—the distance from the baseline to the tops of uppercase characters.
- `float getDescent()`
  gets the font descent—the distance from the baseline to the bottoms of descenders.
- `float getLeading()`
  gets the font leading—the space between the bottom of one line of text and the top of the next line.
- `float getHeight()`
  gets the total height of the font—the distance between the two baselines of text (descent + leading + ascent).

`java.awt.Graphics2D` 1.2

- `FontRenderContext getFontRenderContext()`
  gets a font render context that specifies font characteristics in this graphics context.
- `void drawString(String str, float x, float y)`
  draws a string in the current font and color.

`javax.swing.JComponent` 1.2

- `FontMetrics getFontMetrics(Font f)` 5.0
  gets the font metrics for the given font. The `FontMetrics` class is a precursor to the `LineMetrics` class.

`java.awt.FontMetrics` 1.0

- `FontRenderContext getFontRenderContext()` 1.2
  gets a font render context for the font.

10.3.4. Displaying Images

You can use the `ImageIcon` class to read an image from a file:

```
Image image = new ImageIcon(filename).getImage();
```

Now the variable `image` contains a reference to an object that encapsulates the image data. Display the image with the `drawImage` method of the `Graphics` class.

```
public void paintComponent(Graphics g) {
    . . .
    g.drawImage(image, x, y, null);
}
```

We can take this a little bit further and tile the window with the graphics image. The result looks like the screen shown in **Figure 10.11**. We do the tiling in the `paintComponent` method. We first draw one copy of the image in the top left corner and then use the `copyArea` call to copy it into the entire window:

```
for (int i = 0; i * imageWidth <= getWidth(); i++)
    for (int j = 0; j * imageHeight <= getHeight(); j++)
        if (i + j > 0)
            g.copyArea(0, 0, imageWidth, imageHeight, i * imageWidth, j * imageHeight);
```

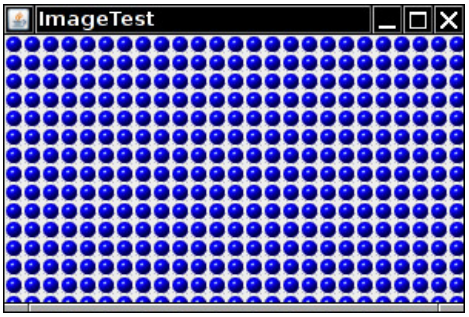


Figure 10.11: Window with tiled graphics image

Listing 10.5 v2ch10/image/ImageDemo.java

```
1  package v2ch10.image;
2
3  import module java.desktop;
4
5  class ImageDemo {
6      void main() {
7          EventQueue.invokeLater(() -> {
8              JFrame frame = new ImageFrame();
9              frame.setTitle("ImageTest");
10             frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
11             frame.setVisible(true);
12         });
13     }
14 }
15
16 /**
17  * A frame with an image component.
18  */
19 class ImageFrame extends JFrame {
20     public ImageFrame() {
21         add(new ImageComponent());
22         pack();
23     }
24 }
25
26 /**
27  * A component that displays a tiled image.
28  */
29 class ImageComponent extends JComponent {
30     private static final int DEFAULT_WIDTH = 300;
31     private static final int DEFAULT_HEIGHT = 200;
32
33     private Image image;
34
35     public ImageComponent() {
36         image = new ImageIcon("images/blue-ball.gif").getImage();
37     }
38
39     public void paintComponent(Graphics g) {
```

```
        if (image == null) return;

        int imageWidth = image.getWidth(null);
        int imageHeight = image.getHeight(null);
        if (imageWidth <= 0 || imageHeight <= 0) return;

        // draw the image in the upper-left corner

        g.drawImage(image, 0, 0, null);
        // tile the image across the component

        for (int i = 0; i * imageWidth <= getWidth(); i++)
            for (int j = 0; j * imageHeight <= getHeight(); j++)
                if (i + j > 0)
                    g.copyArea(0, 0, imageWidth, imageHeight, i * imageWidth, j * imageHeight);
    }

    public Dimension getPreferredSize() {
        return new Dimension(DEFAULT_WIDTH, DEFAULT_HEIGHT);
    }
}
```

`java.awt.Graphics` 1.0

- `boolean drawImage(Image img, int x, int y, ImageObserver observer)`
- `boolean drawImage(Image img, int x, int y, int width, int height, ImageObserver observer)`
  draw an unscaled or scaled image. Note: This call may return before the image is drawn. The `imageObserver` object is notified of the rendering progress. This was a useful feature in the distant past. Nowadays, just pass a `null` observer.
- `void copyArea(int x, int y, int width, int height, int dx, int dy)`
  copies an area of the screen. The `dx` and `dy` parameters are the distance from the source area to the target area.

10.4. Event Handling

Any operating environment that supports GUIs constantly monitors events such as keystrokes or mouse clicks. These events are then reported to the programs that are running. Each program then decides what, if anything, to do in response to these events.

10.4.1. Basic Event Handling Concepts

In the Java AWT, *event sources* (such as buttons or scrollbars) have methods that allow you to register *event listeners*—objects that carry out the desired response to the event.

When an event listener is notified about an event, information about the event is encapsulated in an *event object*. In Java, all event objects ultimately derive from the class `java.util.EventObject.` Of course, there are subclasses for each event type, such as `ActionEvent` and `WindowEvent`.

Different event sources can produce different kinds of events. For example, a button can send `ActionEvent` objects, whereas a window can send `WindowEvent` objects.

To sum up, here's an overview of how event handling in the AWT works:

- An event listener is an instance of a class that implements a *listener interface*.
- An event source is an object that can register listener objects and send them event objects.
- The event source sends out event objects to all registered listeners when that event occurs.
- The listener objects then use the information in the event object to determine their reaction to the event.

Figure 10.12 shows the relationship between the event handling classes and interfaces.

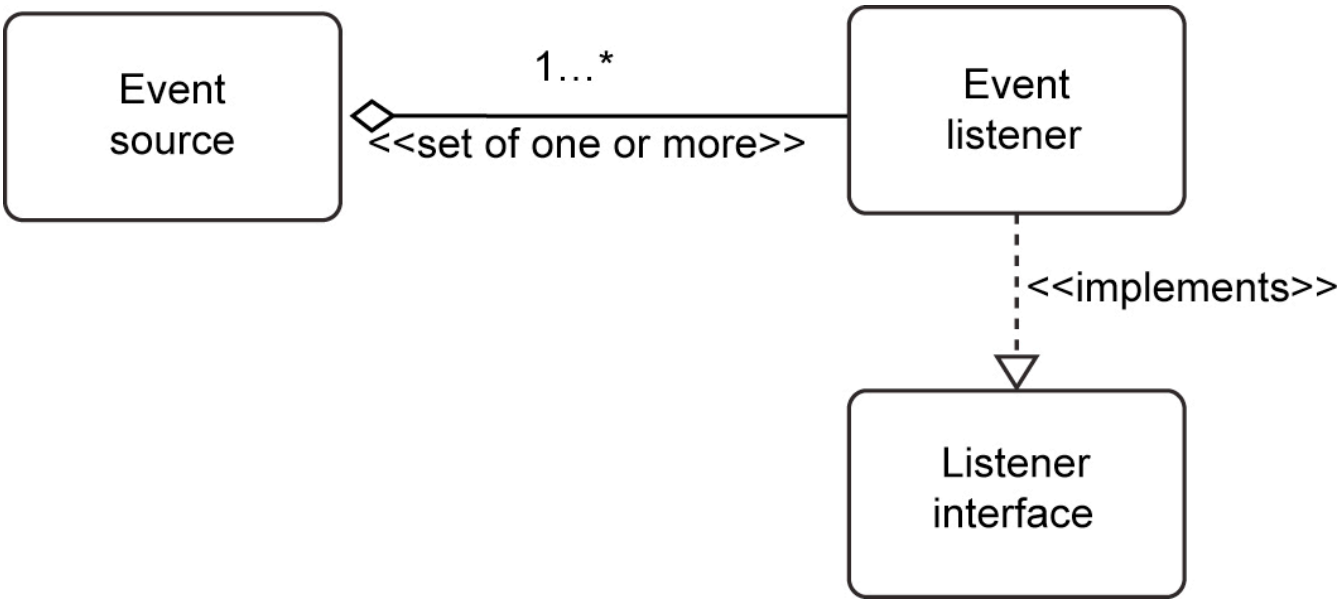


Figure 10.12: Relationship between event sources and listeners

Here is an example for specifying a listener:

```
ActionListener listener = . . .;
var button = new JButton("OK");
button.addActionListener(listener);
```

Now the `listener` object is notified whenever an "action event" occurs in the button. For buttons, as you might expect, an action event is a button click.

To implement the `ActionListener` interface, the listener class must have a method called `actionPerformed` that receives an `ActionEvent` object as a parameter.

```
class MyListener implements ActionListener {
    . . .
    public void actionPerformed(ActionEvent event) {
        // reaction to button click goes here
        . . .
    }
}
```

Whenever the user clicks the button, the `JButton` object creates an `ActionEvent` object and calls `listener.actionPerformed(event)`, passing that event object. An event source such as a button can have multiple listeners. In that case, the button calls the `actionPerformed` methods of all listeners whenever the user clicks the button.

Figure 10.13 shows the interaction between the event source, event listener, and event object.

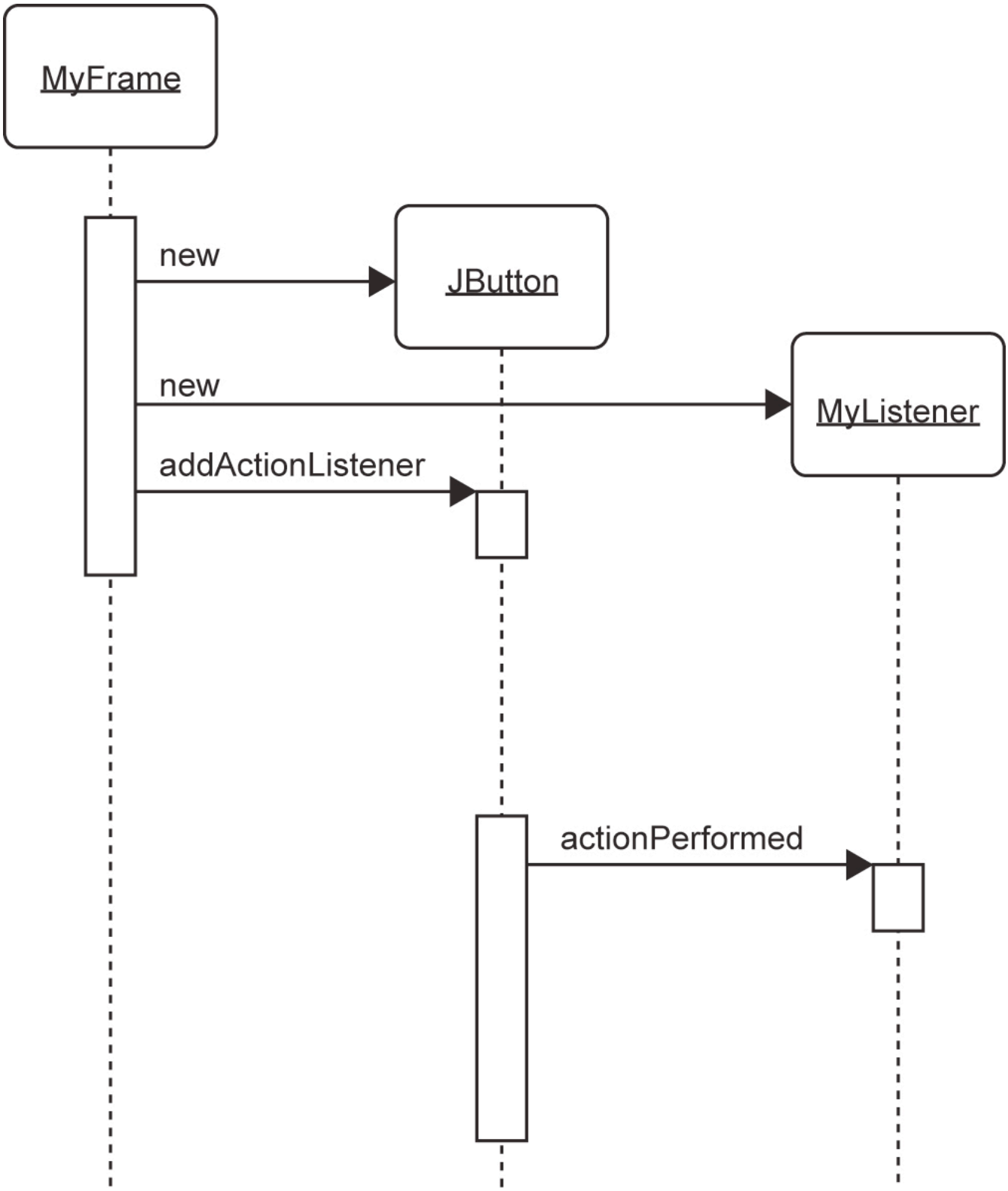


Figure 10.13: Event notification

10.4.2. Example: Handling a Button Click

As a way of getting comfortable with the event delegation model, let's work through all the details needed for the simple example of responding to a button click. For this example, we will show a panel populated with three buttons. Three listener objects are added as action listeners to the buttons.

With this scenario, each time a user clicks on any of the buttons on the panel, the associated listener object receives an `ActionEvent` that indicates a button click. In our sample program, the listener object will then change the background color of the panel.

Before I can show you the program that listens to button clicks, I need to explain how to create buttons and how to add them to a panel.

To create a button, specify a label string, an icon, or both in the button constructor. Here are two examples:

```
var yellowButton = new JButton("Yellow");
var blueButton = new JButton(new ImageIcon("blue-ball.gif"));
```

Call the `add` method to add the buttons to a panel:

```
var yellowButton = new JButton("Yellow");
var blueButton = new JButton("Blue");
var redButton = new JButton("Red");

buttonPanel.add(yellowButton);
buttonPanel.add(blueButton);
buttonPanel.add(redButton);
```

Figure 10.14 shows the result.

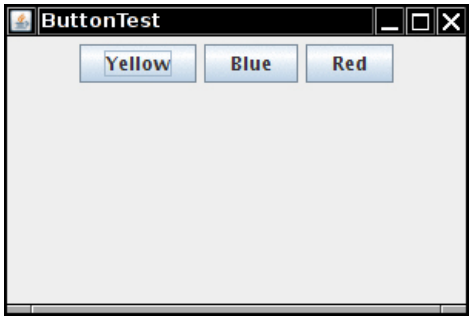


Figure 10.14: A panel filled with buttons

Next, we need to add code that listens to these buttons. This requires classes that implement the `ActionListener` interface, which, as I just mentioned, has one method: `actionPerformed`, whose signature looks like this:

```
public void actionPerformed(ActionEvent event)
```

The way to use the `ActionListener` interface is the same in all situations: The `actionPerformed` method (which is the only method in `ActionListener`) has an object of type `ActionEvent` as a parameter. This event object gives you information about the event that happened.

When a button is clicked, we want the background color of the panel to change to a particular color. We store the desired color in our listener class.

```
class ColorAction implements ActionListener {
    private Color backgroundColor;

    public ColorAction(Color c) {
        backgroundColor = c;
    }

    public void actionPerformed(ActionEvent event) {
        // set panel background color
```

```
        . . .
    }
}
```

We then construct one object for each color and set the objects as the button listeners.

```
var yellowAction = new ColorAction(Color.YELLOW);
var blueAction = new ColorAction(Color.BLUE);
var redAction = new ColorAction(Color.RED);

yellowButton.addActionListener(yellowAction);
blueButton.addActionListener(blueAction);
redButton.addActionListener(redAction);
```

For example, if a user clicks on the button marked “Yellow”, the `actionPerformed` method of the `yellowAction` object is called. Its `backgroundColor` instance field is set to `Color.YELLOW`, and it can now proceed to set the panel’s background color.

Just one issue remains. The `ColorAction` object doesn’t have access to the `buttonPanel` variable. You can solve this problem in two ways. You can store the panel in the `ColorAction` object and set it in the `ColorAction` constructor. Or, more conveniently, you can make `ColorAction` into an inner class of the `ButtonFrame` class. Its methods can then access the outer panel automatically.

Listing 10.6 contains the complete frame class. Whenever you click one of the buttons, the appropriate action listener changes the background color of the panel.

Listing 10.6 v2ch10/button/ButtonFrame.java

```
1  package v2ch10.button;
2
3  import module java.desktop;
4
5  /**
6   * A frame with a button panel.
7   */
8  public class ButtonFrame extends JFrame {
9      private JPanel buttonPanel;
10     private static final int DEFAULT_WIDTH = 300;
11     private static final int DEFAULT_HEIGHT = 200;
12
13     public ButtonFrame() {
14         setSize(DEFAULT_WIDTH, DEFAULT_HEIGHT);
15
16         // create buttons
17         var yellowButton = new JButton("Yellow");
18         var blueButton = new JButton("Blue");
19         var redButton = new JButton("Red");
20
21         buttonPanel = new JPanel();
22
23         // add buttons to panel
24         buttonPanel.add(yellowButton);
25         buttonPanel.add(blueButton);
26         buttonPanel.add(redButton);
```

```
27
28          // add panel to frame
29          add(buttonPanel);
30
31          // create button actions
32          var yellowAction = new ColorAction(Color.YELLOW);
33          var blueAction = new ColorAction(Color.BLUE);
34          var redAction = new ColorAction(Color.RED);
35
36          // associate actions with buttons
37          yellowButton.addActionListener(yellowAction);
38          blueButton.addActionListener(blueAction);
39          redButton.addActionListener(redAction);
40      }
41
42      /**
43       * An action listener that sets the panel's background color.
44       */
45      private class ColorAction implements ActionListener {
46          private Color backgroundColor;
47
48          public ColorAction(Color c) {
49              backgroundColor = c;
50          }
51
52          public void actionPerformed(ActionEvent event) {
53              buttonPanel.setBackground(backgroundColor);
54          }
55      }
56  }
```

`javax.swing.JButton` 1.2

- `JButton(String label)`
- `JButton(Icon icon)`
- `JButton(String label, Icon icon)`
  construct a button. The label string can be plain text or HTML; for example, `"<html><b>Ok</b></html>"`.

`java.awt.Container` 1.0

- `Component add(Component c)`
  adds the component `c` to this container.

10.4.3. Specifying Listeners Concisely

In the preceding section, we defined a class for the event listener and constructed three objects of that class. It is not all that common to have multiple instances of a listener class. Most commonly, each listener carries out a separate action. In that case, there is no need to make a separate class. Simply use a lambda expression:

```
exitButton.addActionListener(_ -> System.exit(0));
```



Note: Recall that the `_` denotes an unused parameter. Prior to Java 22, use a variable name such as `event` instead.

Now consider the case in which we have multiple related actions, such as the color buttons of the preceding section. In such a case, implement a helper method:

```
public void makeButton(String name, Color backgroundColor) {
    var button = new JButton(name);
    buttonPanel.add(button);
    button.addActionListener(_ -> buttonPanel.setBackground(backgroundColor));
}
```

Note that the lambda expression refers to the parameter variable `backgroundColor`.

Then we simply call

```
makeButton("yellow", Color.YELLOW);
makeButton("blue", Color.BLUE);
makeButton("red", Color.RED);
```

Here, we construct three listener objects, one for each color, without explicitly defining a class. Each time the helper method is called, it makes an instance of a class that implements the `ActionListener` interface. Its `actionPerformed` action references the `backGroundColor` value that is, in fact, stored with the listener object. However, all this happens without you having to explicitly define listener classes, instance fields, or constructors that set them.



Note: In older code, you may see the use of anonymous classes:

```
exitButton.addActionListener(new ActionListener() {
    public void actionPerformed(new ActionEvent) {
        System.exit(0);
    }
});
```

Of course, this rather verbose code is no longer necessary. Using a lambda expression is simpler and clearer.

10.4.4. Adapter Classes

Not all events are as simple to handle as button clicks. Suppose you want to monitor when the user tries to close the main frame in order to put up a dialog and exit the program only when the user agrees.

When the user tries to close a window, the `JFrame` object is the source of a `WindowEvent`. If you want to catch that event, you must have an appropriate listener object and add it to the frame's list of window listeners.

```
WindowListener listener = . . .;
frame.addWindowListener(listener);
```

The window listener must be an object of a class that implements the `WindowListener` interface. There are actually seven methods in the `WindowListener` interface. The frame calls them as the responses to seven distinct events that could happen to a window. The names are self-explanatory, except that "iconified" is usually called "minimized" under Windows. Here is the complete `WindowListener` interface:

```
public interface WindowListener {
    void windowOpened(WindowEvent e);
    void windowClosing(WindowEvent e);
    void windowClosed(WindowEvent e);
    void windowIconified(WindowEvent e);
    void windowDeiconified(WindowEvent e);
    void windowActivated(WindowEvent e);
    void windowDeactivated(WindowEvent e);
}
```

Of course, we can define a class that implements the interface, add a call to `System.exit(0)` in the `windowClosing` method, and write do-nothing functions for the other six methods. However, typing code for six methods that don't do anything is the kind of tedious busywork that nobody likes. To simplify this task, each of the AWT listener interfaces that have more than one method comes with a companion *adapter* class that implements all the methods in the interface but does nothing with them. For example, the `WindowAdapter` class has seven do-nothing methods. You extend the adapter class to specify the desired reactions to some, but not all, of the event types in the interface. (An interface such as `ActionListener` that has only a single method does not need an adapter class.)

Here is how we can define a window listener that overrides the `windowClosing` method:

```
class Terminator extends WindowAdapter {
    public void windowClosing(WindowEvent e) {
        if (user agrees)
            System.exit(0);
    }
}
```

Now you can register an object of type `Terminator` as the event listener:

```
var listener = new Terminator();
frame.addWindowListener(listener);
```



Note: Nowadays, one would implement do-nothing methods of the `WindowListener` interface as default methods. However, Swing was invented many years before there were default methods.

java.awt.event.WindowListener **1.1**

- `void windowOpened(WindowEvent e)`
  is called after the window has been opened.
- `void windowClosing(WindowEvent e)`
  is called when the user has issued a window manager command to close the window. Note that the window will close only if its `hide` or `dispose` method is called.
- `void windowClosed(WindowEvent e)`
  is called after the window has closed.

- `void windowIconified(WindowEvent e)`
  is called after the window has been iconified.
- `void windowDeiconified(WindowEvent e)`
  is called after the window has been deiconified.
- `void windowActivated(WindowEvent e)`
  is called after the window has become active. Only a frame or dialog can be active. Typically, the window manager decorates the active window—for example, by highlighting the title bar.
- `void windowDeactivated(WindowEvent e)`
  is called after the window has become deactivated.

java.awt.event.WindowStateListener **1.4**

- `void windowStateChanged(WindowEvent event)`
  is called after the window has been maximized, iconified, or restored to normal size.

10.4.5. Actions

It is common to have multiple ways to activate the same command. The user can choose a certain function through a menu, a keystroke, or a button on a toolbar. This is easy to achieve in the AWT event model: link all events to the same listener. For example, suppose `blueAction` is an action listener whose `actionPerformed` method changes the background color to blue. You can attach the same object as a listener to several event sources:

- A toolbar button labeled “Blue”
- A menu item labeled “Blue”
- A keystroke Ctrl+B

The color change command will now be handled in a uniform way, no matter whether it was caused by a button click, a menu selection, or a key press.

The Swing package provides a very useful mechanism to encapsulate commands and to attach them to multiple event sources: the `Action` interface. An *action* is an object that encapsulates

- A description of the command (as a text string and an optional icon); and
- Parameters that are necessary to carry out the command (such as the requested color in our example).

The `Action` interface has the following methods:

```
void actionPerformed(ActionEvent event)
void setEnabled(boolean b)
boolean isEnabled()
void putValue(String key, Object value)
Object getValue(String key)
void addPropertyChangeListener(PropertyChangeListener listener)
void removePropertyChangeListener(PropertyChangeListener listener)
```

The first method is the familiar method in the `ActionListener` interface; in fact, the `Action` interface extends the `ActionListener` interface. Therefore, you can use an `Action` object whenever an `ActionListener` object is expected.

The next two methods let you enable or disable the action and check whether the action is currently enabled. When an action is attached to a menu or toolbar and the action is disabled, the option is grayed out.

The `putValue` and `getValue` methods let you store and retrieve arbitrary name/value pairs in the action object. A couple of important predefined strings, namely `Action.NAME` and `Action.SMALL_ICON`, store action names and icons into an action object:

```
action.putValue(Action.NAME, "Blue");
action.putValue(Action.SMALL_ICON, new ImageIcon("blue-ball.gif"));
```

Table 10.1 shows all predefined action table names.

Table 10.1: Predefined Action Table Names

Name	Value
`NAME`	The name of the action, displayed on buttons and menu items.
`SMALL_ICON`	A place to store a small icon for display in a button, menu item, or toolbar.
`SHORT_DESCRIPTION`	A short description of the icon for display in a tooltip.
`LONG_DESCRIPTION`	A long description of the icon for potential use in online help. No Swing component uses this value.
`MNEMONIC_KEY`	A mnemonic abbreviation for display in menu items.
`ACCELERATOR_KEY`	A place to store an accelerator keystroke. No Swing component uses this value.
`ACTION_COMMAND_KEY`	Historically, used in the now-obsolete `registerKeyboardAction` method.
`DEFAULT`	Potentially useful catch-all property. No Swing component uses this value.

If the action object is added to a menu or toolbar, the name and icon are automatically retrieved and displayed in the menu item or toolbar button. The `SHORT_DESCRIPTION` value turns into a tooltip.

The final two methods of the `Action` interface allow other objects, in particular menus or toolbars that trigger the action, to be notified when the properties of the action object change. For example, if a menu is added as a property change listener of an action object and the action object is subsequently disabled, the menu is called and can gray out the action name.

Note that `Action` is an *interface*, not a class. Any class implementing this interface must implement the seven methods we just discussed. Fortunately, a friendly soul has provided a class `AbstractAction` that implements all methods except for `actionPerformed`. That class takes care of storing all name/value pairs and managing the property change listeners. You simply extend `AbstractAction` and supply an `actionPerformed` method.

Let's build an action object that can execute color change commands. We store the name of the command, an icon, and the desired color. We store the color in the table of name/value pairs that the `AbstractAction` class provides. Here is the code for the `ColorAction` class. The constructor sets the name/value pairs, and the `actionPerformed` method carries out the color change action.

```
public class ColorAction extends AbstractAction {
    public ColorAction(String name, Icon icon, Color c) {
        putValue(Action.NAME, name);
        putValue(Action.SMALL_ICON, icon);
        putValue("color", c);
        putValue(Action.SHORT_DESCRIPTION, "Set panel color to " + name.toLowerCase());
    }

    public void actionPerformed(ActionEvent event) {
        Color c = (Color) getValue("color");
        buttonPanel.setBackground(c);
    }
}
```

Our test program creates three objects of this class, such as

```
var blueAction = new ColorAction("Blue", new ImageIcon("blue-ball.gif"), Color.BLUE);
```

Next, let's associate this action with a button. That is easy because we can use a `JButton` constructor that takes an `Action` object.

```
var blueButton = new JButton(blueAction);
```

That constructor reads the name and icon from the action, sets the short description as the tooltip, and sets the action as the listener. You can see the icons and a tooltip in **Figure 10.15**.

As demonstrated in the next chapter, it is just as easy to add the same action to a menu.

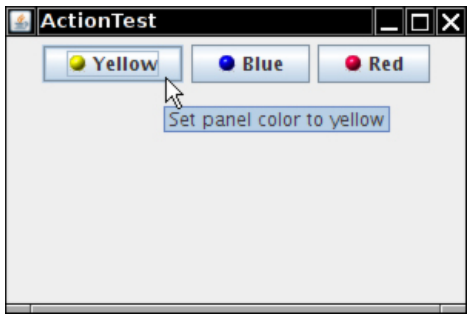


Figure 10.15: Buttons display the icons from the action objects.

Finally, we want to add the action objects to keystrokes so that an action is carried out when the user types a keyboard command. To associate actions with keystrokes, you first need to generate objects of the `KeyStroke` class. This convenience class encapsulates the description of a key. To generate a `KeyStroke` object, don't call a constructor but instead use the static `getKeyStroke` method of the `KeyStroke` class.

```
KeyStroke ctrlBKey = KeyStroke.getKeyStroke("ctrl B");
```

To understand the next step, you need to understand the concept of *keyboard focus*. A user interface can have many buttons, menus, scrollbars, and other components. When you hit a key, it is sent to the component that has focus. That component is usually (but not always) visually distinguished. For example, in the Java look-and-feel, a button with focus has a thin rectangular border around the button text. You can use the Tab key to move the fo-

cus between components. When you press the space bar, the button with focus is clicked. Other keys carry out different actions; for example, the arrow keys can move a scrollbar.

However, in our case, we do not want to send the keystroke to the component that has focus. Otherwise, each of the buttons would need to know how to handle the Ctrl+Y, Ctrl+B, and Ctrl+R keys.

This is a common problem, and the Swing designers came up with a convenient solution. Every `JComponent` has three *input maps*, each mapping `KeyStroke` objects to associated actions. The three input maps correspond to three different conditions (see **Table 10.2**).

Table 10.2: Input Map Conditions

Flag	Invoke Action
`WHEN_FOCUSED`	When this component has keyboard focus
`WHEN_ANCESTOR_OF_FOCUSED_COMPONENT`	When this component contains the component that has keyboard focus
`WHEN_IN_FOCUSED_WINDOW`	When this component is contained in the same window as the component that has keyboard focus

Keystroke processing checks these maps in the following order:

1. Check the `WHEN_FOCUSED` map of the component with input focus. If the keystroke exists and its corresponding action is enabled, execute the action and stop processing.
2. Starting from the component with input focus, check the `WHEN_ANCESTOR_OF_FOCUSED_COMPONENT` maps of its parent components. As soon as a map with the keystroke and a corresponding enabled action is found, execute the action and stop processing.
3. Look at all *visible* and *enabled* components, in the window with input focus, that have this keystroke registered in a `WHEN_IN_FOCUSED_WINDOW` map. Give these components (in the order of their keystroke registration) a chance to execute the corresponding action. As soon as the first enabled action is executed, stop processing.

To obtain an input map from the component, use the `getInputMap` method. Here is an example:

```
InputMap imap = panel.getInputMap(JComponent.WHEN_FOCUSED);
```

The `WHEN_FOCUSED` condition means that this map is consulted when the current component has the keyboard focus. In our situation, that isn't the map we want. One of the buttons, not the panel, has the input focus. Either of the other two map choices works fine for inserting the color change keystrokes. We use `WHEN_ANCESTOR_OF_FOCUSED_COMPONENT` in our example program.

The `InputMap` doesn't directly map `KeyStroke` objects to `Action` objects. Instead, it maps to arbitrary objects, and a second map, implemented by the `ActionMap` class, maps objects to actions. That makes it easier to share the same actions among keystrokes that come from different input maps.

Thus, each component has three input maps and one action map. To tie them together, you need to come up with names for the actions. Here is how you can tie a key to an action:

```
imap.put(KeyStroke.getKeyStroke("ctrl Y"), "panel.yellow");
ActionMap amap = panel.getActionMap();
amap.put("panel.yellow", yellowAction);
```

It is customary to use the string `"none"` for a do-nothing action. That makes it easy to deactivate a key:

```
imap.put(KeyStroke.getKeyStroke("ctrl C"), "none");
```



Caution: The JDK documentation suggests using the action name as the action's key. I don't think that is a good idea. The action name is displayed on buttons and menu items; thus, it can change at the whim of the UI designer and may be translated into multiple languages. Such unstable strings are poor choices for lookup keys, so I recommend that you come up with action names that are independent of the displayed names.

To summarize, here is what you do to carry out the same action in response to a button, a menu item, or a keystroke:

1. Implement a class that extends the `AbstractAction` class. You may be able to use the same class for multiple related actions.
2. Construct an object of the action class.
3. Construct a button or menu item from the action object. The constructor will read the label text and icon from the action object.
4. For actions that can be triggered by keystrokes, you have to carry out additional steps. First, locate the top-level component of the window, such as a panel that contains all other components.
5. Then, get the `WHEN_ANCESTOR_OF_FOCUSED_COMPONENT` input map of the top-level component. Make a `KeyStroke` object for the desired keystroke. Make an action key object, such as a string that describes your action. Add the pair (keystroke, action key) into the input map.
6. Finally, get the action map of the top-level component. Add the pair (action key, action object) into the map.

javax.swing.Action **1.2**

- `boolean isEnabled()`
- `void setEnabled(boolean b)`
  get or set the `enabled` property of this action.
- `void putValue(String key, Object value)`
  places a key/value pair inside the action object. The key can be any string, but several names have predefined meanings—see .
- `Object getValue(String key)`
  returns the value of a stored name/value pair.

`javax.swing.KeyStroke` **1.2**

- `static KeyStroke getKeyStroke(String description)`
  constructs a keystroke from a human-readable description (a sequence of whitespace-delimited strings). The description starts with zero or more modifiers (`shift`, `control`, `ctrl`, `meta`, `alt`, `altGraph`) and ends with either the string `typed`, followed by a one-character string (for example, `"typed a"`), or an optional event specifier (`pressed` or `released`, with `pressed` being the default), followed by a key

code. The key code, when prefixed with `VK_`, should correspond to a `KeyEvent` constant; for example, `"INSERT"` corresponds to `KeyEvent.VK_INSERT`.

`javax.swing.JComponent` 1.2

- `ActionMap getActionMap()` 1.3
  returns the map that associates action map keys (which can be arbitrary objects) with `Action` objects.
- `InputMap getInputMap(int flag)` 1.3
  gets the input map that maps key strokes to action map keys. The flag is one of the values in .

10.4.6. Mouse Events

You do not need to handle mouse events explicitly if you just want the user to be able to click on a button or menu. These mouse operations are handled internally by the various components in the user interface. However, if you want to enable the user to draw with the mouse, you will need to trap the mouse move, click, and drag events.

In this section, I will show you a simple graphics editor application that allows the user to place, move, and erase squares on a canvas (see **Figure 10.16**).

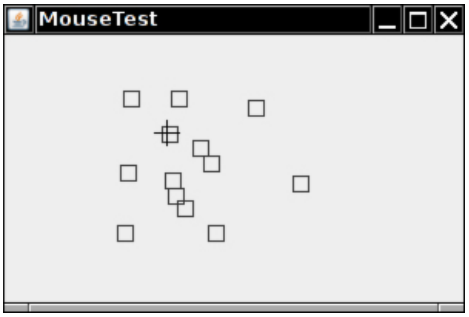


Figure 10.16: A mouse test program

When the user clicks a mouse button, three listener methods are called: `mousePressed` when the mouse is first pressed, `mouseReleased` when the mouse is released, and, finally, `mouseClicked`. If you are only interested in complete clicks, you can ignore the first two methods. By using the `getX` and `getY` methods on the `MouseEvent` parameter, you can obtain the *x* and *y* coordinates of the mouse pointer when the mouse was clicked. To distinguish between single, double, and triple (!) clicks, use the `getClickCount` method.

In our sample program, we supply both a `mousePressed` and a `mouseClicked` method. When you click on a pixel that is not inside any of the squares that have been drawn, a new square is added. We implemented this in the `mousePressed` method so that the user receives immediate feedback and does not have to wait until the mouse button is released. When a user double-clicks inside an existing square, it is erased. We implemented this in the `mouseClicked` method because we need the click count.

```
public void mousePressed(MouseEvent event) {
    current = find(event.getPoint());
    if (current == null) // not inside a square
        add(event.getPoint());
}

public void mouseClicked(MouseEvent event) {
    current = find(event.getPoint());
```

```
    if (current != null && event.getClickCount() >= 2)
        remove(current);
}
```

As the mouse moves over a window, the window receives a steady stream of mouse movement events. Note that there are separate `MouseListener` and `MouseMotionListener` interfaces. This is done for efficiency—there are a lot of mouse events as the user moves the mouse around, and a listener that just cares about mouse *clicks* will not be bothered with unwanted mouse *moves*.

Our test application traps mouse motion events to change the cursor to a different shape (a cross hair) when it is over a square. This is done with the `getPredefinedCursor` method of the `Cursor` class. **Table 10.3** lists the constants to use with this method along with what the cursors look like under Windows.

Table 10.3: Sample Cursor Shapes

Icon	Constant	Icon	Constant
	`DEFAULT_CURSOR`		`NE_RESIZE_CURSOR`
	`CROSSHAIR_CURSOR`		`E_RESIZE_CURSOR`
	`HAND_CURSOR`		`SE_RESIZE_CURSOR`
	`MOVE_CURSOR`		`S_RESIZE_CURSOR`
	`TEXT_CURSOR`		`SW_RESIZE_CURSOR`
	`WAIT_CURSOR`		`W_RESIZE_CURSOR`
	`N_RESIZE_CURSOR`		`NW_RESIZE_CURSOR`

Here is the `mouseMoved` method of the `MouseMotionListener` in our example program:

```
public void mouseMoved(MouseEvent event) {
    if (find(event.getPoint()) == null)
        setCursor(Cursor.getDefaultCursor());
    else
        setCursor(Cursor.getPredefinedCursor(Cursor.CROSSHAIR_CURSOR));
}
```

If the user presses a mouse button while the mouse is in motion, `mouseDragged` calls are generated instead of `mouseMoved` calls. Our test application lets a user drag the square under the cursor. We simply update the currently dragged rectangle to be centered under the mouse position. Then, we repaint the canvas to show the new mouse position.

```
public void mouseDragged(MouseEvent event) {
    if (current != null) {
        int x = event.getX();
        int y = event.getY();

        current.setFrame(x - SIDELENGTH / 2, y - SIDELENGTH / 2, SIDELENGTH, SIDELENGTH);
        repaint();
    }
}
```



Note: The `mouseMoved` method is only called as long as the mouse stays inside the component. However, the `mouseDragged` method keeps getting called even when the mouse is being dragged outside the component.

There are two other mouse event methods: `mouseEntered` and `mouseExited`. These methods are called when the mouse enters or exits a component.

Finally, let's see how to listen to mouse events. Mouse clicks are reported through the `mouseClicked` method, which is part of the `MouseListener` interface. Many applications are only interested in mouse clicks and not in mouse moves; with the mouse move events occurring so frequently, the mouse move and drag events are defined in a separate interface called `MouseMotionListener`.

In our program we are interested in both types of mouse events. We define two inner classes: `MouseHandler` and `MouseMotionHandler`. The `MouseHandler` class extends the `MouseAdapter` class because it defines only two of the five `MouseListener` methods. (The `MouseAdapter` class defines all of them as do-nothing methods.) The `MouseMotionHandler` implements the `MouseMotionListener` and defines both methods of that interface. **Listing 10.7** is the program listing.

Listing 10.7 v2ch10/mouse/MouseComponent.java

```
1  package v2ch10.mouse;
2
3  import module java.base;
4  import module java.desktop;
5  import java.awt.event.MouseEvent;
6
7  /**
8   * A component with mouse operations for adding and removing squares.
9   */
10 public class MouseComponent extends JComponent {
11     private static final int DEFAULT_WIDTH = 300;
12     private static final int DEFAULT_HEIGHT = 200;
13
14     private static final int SIDELENGTH = 10;
15     private ArrayList<Rectangle2D> squares;
16     private Rectangle2D current; // the square containing the mouse cursor
17
18     public MouseComponent() {
```

```
        squares = new ArrayList<>();
        current = null;

        addMouseListener(new MouseHandler());
        addMouseMotionListener(new MouseMotionHandler());
    }

    public Dimension getPreferredSize() {
        return new Dimension(DEFAULT_WIDTH, DEFAULT_HEIGHT);
    }

    public void paintComponent(Graphics g) {
        var g2 = (Graphics2D) g;

        // draw all squares
        for (Rectangle2D r : squares)
            g2.draw(r);
    }

    /**
     * Finds the first square containing a point.
     * @param p a point
     * @return the first square that contains p
     */
    public Rectangle2D find(Point2D p) {
        for (Rectangle2D r : squares) {
            if (r.contains(p)) return r;
        }
        return null;
    }

    /**
     * Adds a square to the collection.
     * @param p the center of the square
     */
    public void add(Point2D p) {
        double x = p.getX();
        double y = p.getY();

        current = new Rectangle2D.Double(x - SIDELENGTH / 2, y - SIDELENGTH / 2, SIDELENGTH,
                SIDELENGTH);
        squares.add(current);
        repaint();
    }

    /**
     * Removes a square from the collection.
     * @param s the square to remove
     */
    public void remove(Rectangle2D s) {
        if (s == null) return;
        if (s == current) current = null;
        squares.remove(s);
        repaint();
    }

    private class MouseHandler extends MouseAdapter {
        public void mousePressed(MouseEvent event) {
```

```
            // add a new square if the cursor isn't inside a square
            current = find(event.getPoint());
            if (current == null) add(event.getPoint());
        }

        public void mouseClicked(MouseEvent event) {
            // remove the current square if double clicked
            current = find(event.getPoint());
            if (current != null && event.getClickCount() >= 2) remove(current);
        }
    }

    private class MouseMotionHandler implements MouseMotionListener {
        public void mouseMoved(MouseEvent event) {
            // set the mouse cursor to cross hairs if it is inside a rectangle

            if (find(event.getPoint()) == null)
                setCursor(Cursor.getDefaultCursor());
            else
                setCursor(Cursor.getPredefinedCursor(Cursor.CROSSHAIR_CURSOR));
        }

        public void mouseDragged(MouseEvent event) {
            if (current != null) {
                int x = event.getX();
                int y = event.getY();

                // drag the current rectangle to center it at (x, y)
                current.setFrame(x - SIDELENGTH / 2, y - SIDELENGTH / 2, SIDELENGTH,
                        SIDELENGTH);
                repaint();
            }
        }
    }
}
```

`java.awt.event.MouseEvent` **1.1**

- `int getX()`
- `int getY()`
- `Point getPoint()`
  return the *x* (horizontal) and *y* (vertical) coordinates of the point where the event happened, measured from the top left corner of the component that is the event source.
- `int getClickCount()`
  returns the number of consecutive mouse clicks associated with this event. (The time interval for what constitutes “consecutive” is system-dependent.)

`java.awt.Component` **1.0**

- `public void setCursor(Cursor cursor)` **1.1**
  sets the cursor image to the specified cursor.

10.4.7. The AWT Event Hierarchy

The `EventObject` class has a subclass `AWTEvent`, which is the parent of all AWT event classes. **Figure 10.17** shows the inheritance diagram of the AWT events.

Figure 10.17: Inheritance diagram of AWT event classes

Some of the Swing components generate event objects of yet more event types; these directly extend `EventObject`, not `AWTEvent`.

The event objects encapsulate information about the event that the event source communicates to its listeners. When necessary, you can then analyze the event objects that were passed to the listener object, as we did in the button example with the `getSource` and `getActionCommand` methods.

Some of the AWT event classes are of no practical use for the Java programmer. For example, the AWT inserts `PaintEvent` objects into the event queue, but these objects are not delivered to listeners. Java programmers

don't listen to paint events; instead, they override the `paintComponent` method to control repainting. The AWT also generates a number of events that are needed only by systems programmers, to provide input systems for ideographic languages, automated testing robots, and so on.

The AWT makes a useful distinction between *low-level* and *semantic* events. A semantic event is one that expresses what the user is doing, such as "clicking that button"; an `ActionEvent` is a semantic event. Low-level events are those events that make this possible. In the case of a button click, this is a mouse down, a series of mouse moves, and a mouse up (but only if the mouse up is inside the button area). Or it might be a keystroke, which happens if the user selects the button with the Tab key and then activates it with the space bar. Similarly, adjusting a scrollbar is a semantic event, but dragging the mouse is a low-level event.

Here are the most commonly used semantic event classes in the `java.awt.event` package:

- `ActionEvent` (for a button click, a menu selection, selecting a list item, or Enter typed in a text field)
- `AdjustmentEvent` (the user adjusted a scrollbar)
- `ItemEvent` (the user made a selection from a set of checkbox or list items)

Five low-level event classes are commonly used:

- `KeyEvent` (a key was pressed or released)
- `MouseEvent` (the mouse button was pressed, released, moved, or dragged)
- `MouseWheelEvent` (the mouse wheel was rotated)
- `FocusEvent` (a component got focus or lost focus)
- `WindowEvent` (the window state changed)

Table 10.4 shows the most important AWT listener interfaces, events, and event sources.

Table 10.4: Event Handling Summary

Interface	Methods	Parameter/Accessors	Events Generated By

`ActionListener`	`actionPerformed`	`ActionEvent`
		• `getActionCommand`
		• `getModifiers`
			`AbstractButton`
			`JComboBox`
			`JTextField`
			`Timer`

`AdjustmentListener`	`adjustmentValueChanged`	`AdjustmentEvent` • `getAdjustable` • `getAdjustmentType` • `getValue`	`JScrollbar`
`ItemListener`	`itemStateChanged`	`ItemEvent` • `getItem` • `getItemSelectable` • `getStateChange`	`AbstractButton` `JComboBox`

Interface	Methods	Parameter/Accessors	Events Generated By
FocusListener	focusGained focusLost	FocusEvent • isTemporary	Component
KeyListener	keyPressed keyReleased keyTyped	KeyEvent • getKeyChar • getKeyCode • getKeyModifiersText • getKeyText • isActionKey	Component
MouseListener	mousePressed mouseReleased mouseEntered mouseExited mouseClicked	MouseEvent • getClickCount • getX • getY • getPoint • translatePoint	Component
MouseMotionListener	mouseDragged mouseMoved	MouseEvent	Component
MouseWheelListener	mouseWheelMoved	MouseWheelEvent • getWheelRotation • getScrollAmount	Component
WindowListener	windowClosing windowOpened windowIconified windowDeiconified windowClosed windowActivated windowDeactivated	WindowEvent • getWindow	Window
WindowFocusListener	windowGainedFocus windowLostFocus	WindowEvent • getOppositeWindow	Window

Interface	Methods	Parameter/Accessors	Events Generated By
		`WindowEvent`	
`WindowStateListener`	`windowStateChanged`	• `getOldState` • `getNewState`	`Window`

10.5. The Preferences API

I end this chapter with a discussion of the `java.util.prefs` API. In a desktop program, you will often want to store user preferences, such as the last file that the user worked on, the last window location, and so on.

As you have seen in **Chapter 9 of Volume I**, the `Properties` class makes it simple to load and save configuration information of a program. However, using property files has these disadvantages:

- Some operating systems have no concept of a home directory, making it difficult to find a uniform location for configuration files.
- There is no standard convention for naming configuration files, increasing the likelihood of name clashes as users install multiple Java applications.

Some operating systems have a central repository for configuration information. The best-known example is the registry in Microsoft Windows. The `Preferences` class provides such a central repository in a platform-independent manner. In Windows, the `Preferences` class uses the registry for storage; on Linux, the information is stored in the local file system instead. Of course, the repository implementation is transparent to the programmer using the `Preferences` class.

The `Preferences` repository has a tree structure, with node path names such as `/com/mycompany/myapp.` As with package names, name clashes are avoided as long as programmers start the paths with reversed domain names. In fact, the designers of the API suggest that the configuration node paths match the package names in your program.

Each node in the repository has a separate table of key/value pairs that you can use to store numbers, strings, or byte arrays. No provision is made for storing serializable objects. The API designers felt that the serialization format is too fragile for long-term storage. Of course, if you disagree, you can save serialized objects in byte arrays.

For additional flexibility, there are multiple parallel trees. Each program user has one tree; an additional tree, called the system tree, is available for settings that are common to all users. The `Preferences` class uses the operating system's notion of the "current user" for accessing the appropriate user tree.

To access a node in the tree, start with the user or system root:

```
Preferences root = Preferences.userRoot();
```

or

```
Preferences root = Preferences.systemRoot();
```

Then access the node. You can simply provide a node path name:

```
Preferences node = root.node("/com/mycompany/myapp");
```

A convenient shortcut gets a node whose path name equals the package name of a class. Simply take an object of that class and call

```
Preferences node = Preferences.userNodeForPackage(obj.getClass());
```

or

```
Preferences node = Preferences.systemNodeForPackage(obj.getClass());
```

Typically, `obj` will be the `this` reference.

Once you have a node, you can access the key/value table with methods

```
String get(String key, String def)
int getInt(String key, int def)
long getLong(String key, long def)
float getFloat(String key, float def)
double getDouble(String key, double def)
boolean getBoolean(String key, boolean def)
byte[] getByteArray(String key, byte[] def)
```

Note that you must specify a default value when reading the information, in case the repository data is not available. Defaults are required for several reasons. The data might be missing because the user never specified a preference. Certain resource-constrained platforms might not have a repository, and mobile devices might be temporarily disconnected from the repository.

Conversely, you can write data to the repository with `put` methods such as

```
put(String key, String value)
putInt(String key, int value)
```

and so on.

You can enumerate all keys stored in a node with the method

```
String[] keys()
```

There is currently no way to find out the type of the value of a particular key.



Note: Node names and keys are limited to 80 characters, and string values to 8,192 characters.

Central repositories such as the Windows registry traditionally suffer from two problems:

- They turn into a “dumping ground” filled with obsolete information.
- Configuration data gets entangled into the repository, making it difficult to move preferences to a new platform.

The `Preferences` class has a solution for the second problem. You can export the preferences of a subtree (or, less commonly, a single node) by calling the methods

```
void exportSubtree(OutputStream out)
void exportNode(OutputStream out)
```

The data are saved in XML format. You can import them into another repository by calling

```
void importPreferences(InputStream in)
```

Here is a sample file:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE preferences SYSTEM "http://java.sun.com/dtd/preferences.dtd">
<preferences EXTERNAL_XML_VERSION="1.0">
   <root type="user">
      <map/>
      <node name="com">
         <map/>
         <node name="horstmann">
            <map/>
            <node name="corejava">
               <map>
            <entry key="height" value="200.0"/>
            <entry key="left" value="1027.0"/>
            <entry key="filename" value="/home/cay/books/cj11/code/v1ch11/raven.html"/>
            <entry key="top" value="380.0"/>
            <entry key="width" value="300.0"/>
               </map>
            </node>
        </node>
      </node>
   </root>
</preferences>
```

If your program uses preferences, you should give your users the opportunity of exporting and importing them, so they can easily migrate their settings from one computer to another. The program in **Listing 10.8** demonstrates this technique. The program simply saves the window location and the last loaded filename. Try resizing the window, then export your preferences, move the window, exit, and restart the application. The window will be just like you left it when you exited. Import your preferences, and the window reverts to its prior location.

Listing 10.8 v2ch10/preferences/ImageViewer.java

```
1 package v2ch10.preferences;
2
3 import module java.base;
4 import module java.desktop;
5 import module java.prefs;
```

```
6
7  /**
8   * A program to test preference settings. The program remembers the frame position, size, and
9   * last selected file.
10  */
11 class ImageViewer {
12     void main() {
13         EventQueue.invokeLater(() -> {
14             var frame = new ImageViewerFrame();
15             frame.setTitle("ImageViewer");
16             frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
17             frame.setVisible(true);
18         });
19     }
20 }
21
22 /**
23  * An image viewer that restores position, size, and image from user preferences and updates
24  * the preferences upon exit.
25  */
26 class ImageViewerFrame extends JFrame {
27     private static final int DEFAULT_WIDTH = 300;
28     private static final int DEFAULT_HEIGHT = 200;
29     private String image;
30
31     public ImageViewerFrame() {
32         Preferences root = Preferences.userRoot();
33         Preferences node = root.node("/com/horstmann/corejava/ImageViewer");
34         // get position, size, title from properties
35         int left = node.getInt("left", 0);
36         int top = node.getInt("top", 0);
37         int width = node.getInt("width", DEFAULT_WIDTH);
38         int height = node.getInt("height", DEFAULT_HEIGHT);
39         setBounds(left, top, width, height);
40         image = node.get("image", null);
41         var label = new JLabel();
42         if (image != null) label.setIcon(new ImageIcon(image));
43
44         addWindowListener(new WindowAdapter() {
45             public void windowClosing(WindowEvent event) {
46                 node.putInt("left", getX());
47                 node.putInt("top", getY());
48                 node.putInt("width", getWidth());
49                 node.putInt("height", getHeight());
50                 if (image != null) node.put("image", image);
51             }
52         });
53
54         // use a label to display the images
55         add(label);
56
57         // set up the file chooser
58         var chooser = new JFileChooser();
59         chooser.setCurrentDirectory(new File("."));
60
61         // set up the menu bar
62         var menuBar = new JMenuBar();
63         setJMenuBar(menuBar);
```

```
64
65          var menu = new JMenu("File");
66          menuBar.add(menu);
67
68          var openItem = new JMenuItem("Open");
69          menu.add(openItem);
70          openItem.addActionListener(_ -> {
71              // show file chooser dialog
72              int result = chooser.showOpenDialog(null);
73
74              // if file selected, set it as icon of the label
75              if (result == JFileChooser.APPROVE_OPTION) {
76                  image = chooser.getSelectedFile().getPath();
77                  label.setIcon(new ImageIcon(image));
78              }
79          });
80
81          var exitItem = new JMenuItem("Exit");
82          menu.add(exitItem);
83          exitItem.addActionListener(_ -> System.exit(0));
84      }
85  }
```

`java.util.prefs.Preferences` 1.4

- `Preferences userRoot()`
  returns the root preferences node of the user of the calling program.
- `Preferences systemRoot()`
  returns the systemwide root preferences node.
- `Preferences node(String path)`
  returns a node that can be reached from the current node by the given path. If `path` is absolute (that is, starts with a `/`), then the node is located starting from the root of the tree containing this preference node. If there isn't a node with the given path, it is created.
- `Preferences userNodeForPackage(Class cl)`
- `Preferences systemNodeForPackage(Class cl)`
  return a node in the current user's tree or the system tree whose absolute node path corresponds to the package name of the class `cl`.
- `String[] keys()`
  returns all keys belonging to this node.
- `String get(String key, String def)`
- `int getInt(String key, int def)`
- `long getLong(String key, long def)`
- `float getFloat(String key, float def)`
- `double getDouble(String key, double def)`
- `boolean getBoolean(String key, boolean def)`
- `byte[] getByteArray(String key, byte[] def)`
  return the value associated with the given key—or the supplied default value if no value is associated with the key, the associated value is not of the correct type, or the preferences store is unavailable.

- `void put(String key, String value)`
- `void putInt(String key, int value)`
- `void putLong(String key, long value)`
- `void putFloat(String key, float value)`
- `void putDouble(String key, double value)`
- `void putBoolean(String key, boolean value)`
- `void putByteArray(String key, byte[] value)`
  store a key/value pair with this node.
- `void exportSubtree(OutputStream out)`
  writes the preferences of this node and its children to the specified stream.
- `void exportNode(OutputStream out)`
  writes the preferences of this node (but not its children) to the specified stream.
- `void importPreferences(InputStream in)`
  imports the preferences contained in the specified stream.

This concludes our introduction into graphical user interface programming. The next chapter shows you how to work with the most common Swing components.

Previous chapter
< 9. Security

Next chapter
11. User Interface Components with Swing >